System Models

notes.arpanadhikari7.com.np

Introduction to System Modeling

- System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system.
- System modeling has now come to mean representing a system using some kind of graphical notation, which is now almost always based on notations in the Unified Modeling Language (UML).
- A System Model represent aspects of a system and its environment.
- System modelling:
  - helps the analyst to understand the functionality of the system
  - models are used to communicate with customers.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Introduction to System Modeling

- To construct a model, the engineers should consider a number of restraining factors:

1. Assumptions
2. Simplifications
3. Limitations
4. Constraints
5. Preferences

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Existing and Planned System Models

- Models of the existing system are used during requirements engineering.
  - They help clarify what the existing system does and can be used as a basis for discussing its strengths and weaknesses.
  - These then lead to requirements for the new system.
- Models of the new system are used during requirements engineering.
  - System Models help to explain the proposed requirements to other system stakeholders.
  - Engineers use these models to discuss design proposals and to document the system for implementation.
- In a model-driven engineering process, it is possible to generate a complete or partial system implementation from the system model.

Available on notes.arpanadhikari7.com.np

Features of System Model

1. System models define the processes that serve the needs of the view under consideration.
2. System models represent the behavior of the processes and the assumptions on which the behavior is based.
3. System Models explicitly define both a exogenous and endogenous input to the model.
4. System Models represent all linkages(input/output) that will enable engineer to better understand the view.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Need of System Modeling

1. Models help us to visualize a system as it is or as we want it to be.
2. Models permit us to specify the structure or behavior of a system.
3. Models gives us a template that guides us in constructing a system.
4. Models document the decisions we have made.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Context Models

A context model shows:

* What is outside the system
* How the system interacts with external entities

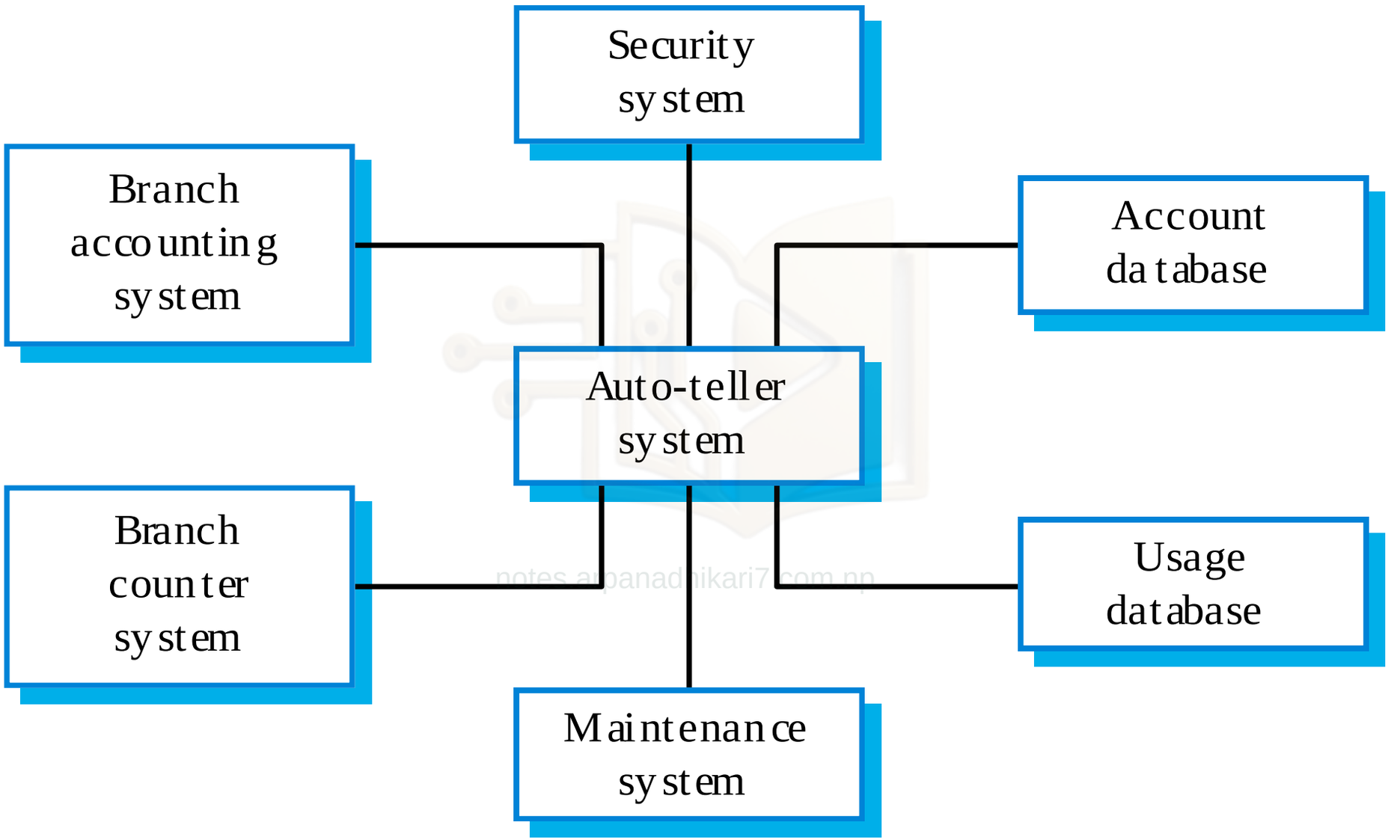
👉 It defines system boundary.

- Context models are used to illustrate the operational context of a system - they show what lies outside the system boundaries.
- System boundaries are established to define what is inside and what is outside the system.
- They show other systems that are used or depend on the system being developed.
- Social and organizational concerns may affect the decision on where to position system boundaries.
- The position of the system boundary has a profound effect on the system requirements.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Context Model of ATM System



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Data Flow Diagram(DFD)

- Data flow diagrams (DFD) graphically illustrate movement of data between external entities and the processes and data stores within a system.
- A data flow diagram (DFD) maps out the flow of information for any process or system.
- Each process transforms inputs into outputs.
- Dataflow models show the flow of information through a system.
- It uses defined symbols data inputs, outputs, storage points and the routes between each destination.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Why use DFD?

- Represents the logical information flow of the system
- Helps in determining physical system construction requirements.
- Simplicity of notation
- Establishment of manual and automated system requirements

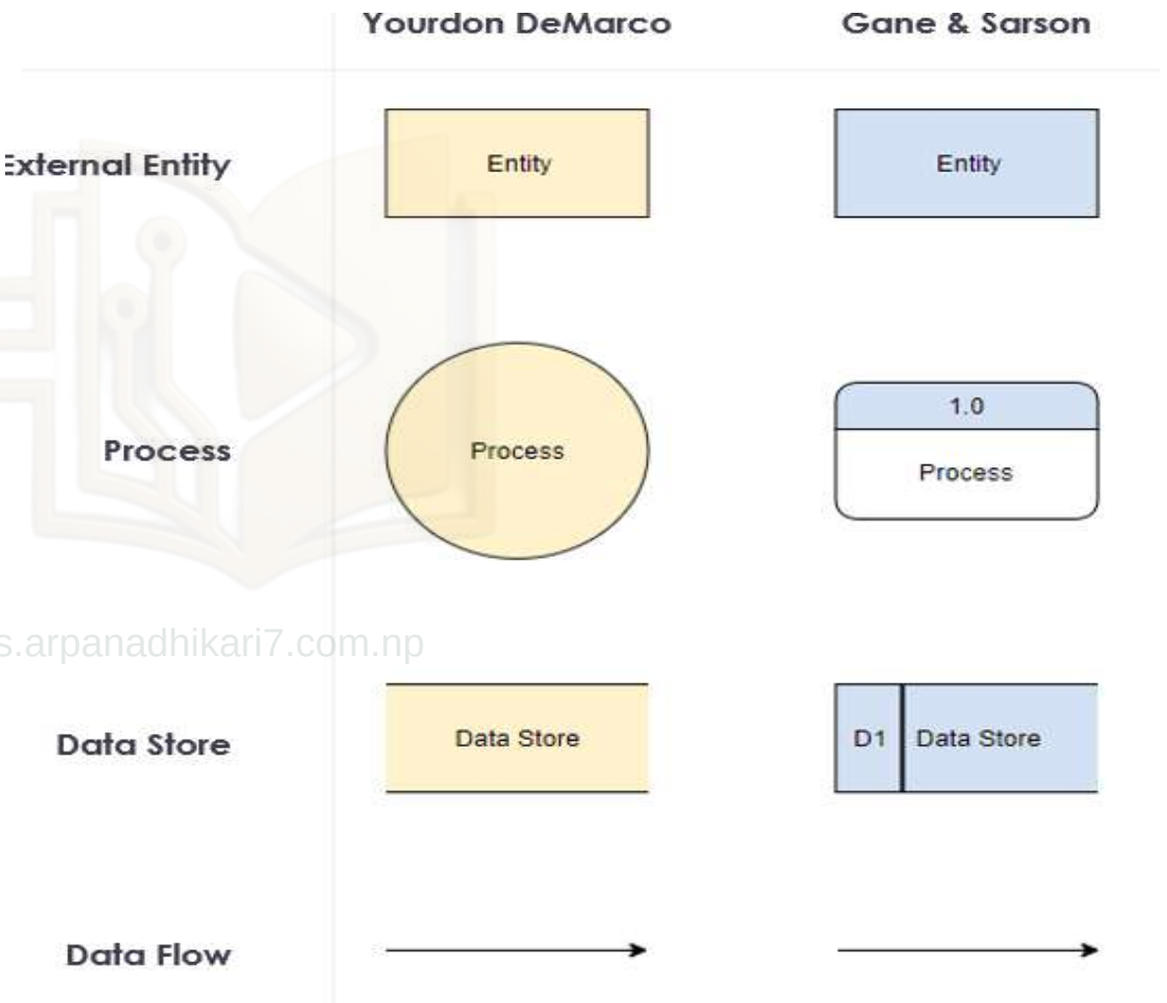
notes.arpanadhikari7.com.np

Symbols and Notations Used in DFDs

➢Two common systems of symbols are named after their creators:

a. Yourdon/DeMarco DFD

b. Gane and Sarson DFD

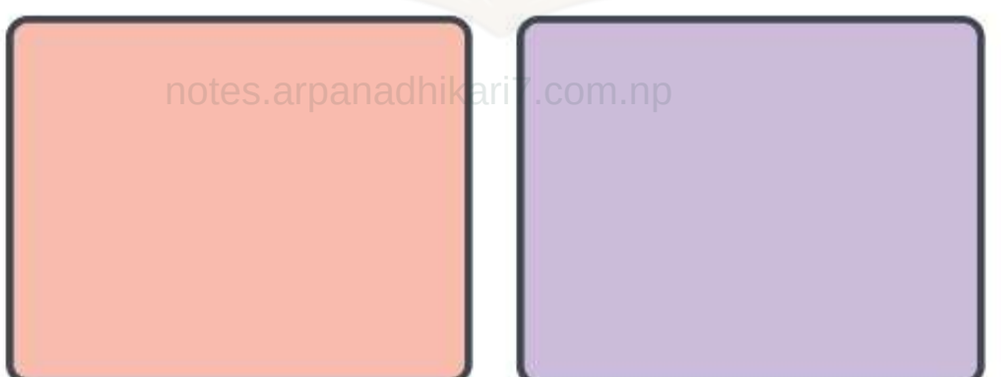


notes.arpanadhikari7.com.np

Components of DFD

1. External entity

- An outside system that sends or receives data, communicating with the system being diagrammed.
- They are the sources and destinations of information entering or leaving the system.
- They might be an outside organization or person, a computer system or a business system.
- They are also known as terminators, sources and sinks or actors.
- They are typically drawn on the edges of the diagram.

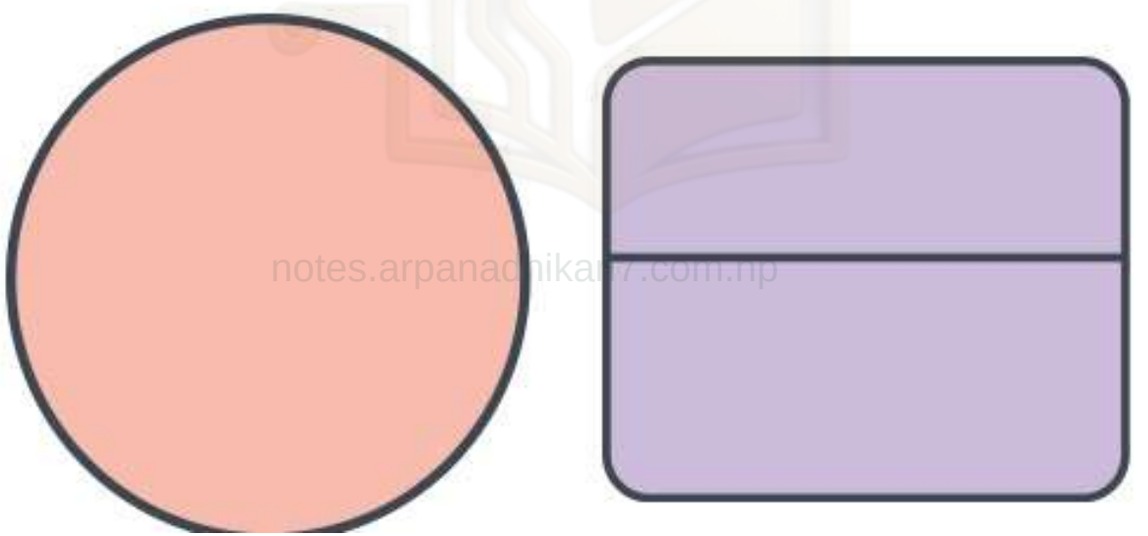


notes.arpanadhikari7.com.np

Components of DFD

2. Process

- Any process that changes the data, producing an output.
- It might perform computations, or sort data based on logic, or direct the data flow based on business rules.
- A short label is used to describe the process, such as “Submit payment.”



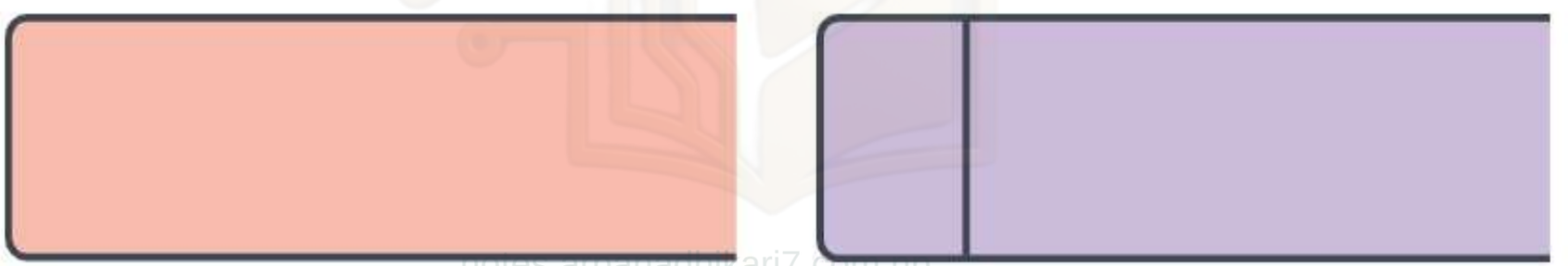
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Components of DFD

3. Data store

- Files or repositories that hold information for later use, such as a database table or a membership form.
- Each data store receives a simple label, such as “Orders.”



notes.arpanadhikari7.com.np

Components of DFD

4. Data flow

- The route that data takes between the external entities, processes and data stores.
- It portrays the interface between the other components and is shown with arrows, typically labeled with a short data name, like “Billing details.”



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Logical vs Physical Data Flow Diagrams

Logical DFD

- A logical data flow diagram focuses on the business and how the business operates.
- It is not concerned with how the system will be constructed.
- We can ignore implementation specifics such as, computer configuration, data storage technology, communication or message passing methods by focusing on the functions performed by the system, such as, data collection, data to information transformation and information reporting.

Physical DFD

notes.arpanadhikari7.com.np

- A physical data flow diagram shows how the system will be implemented, including the hardware, software, files, and people in the system.

Available on notes.arpanadhikari7.com.np

Major Differences between logical and physical dfds

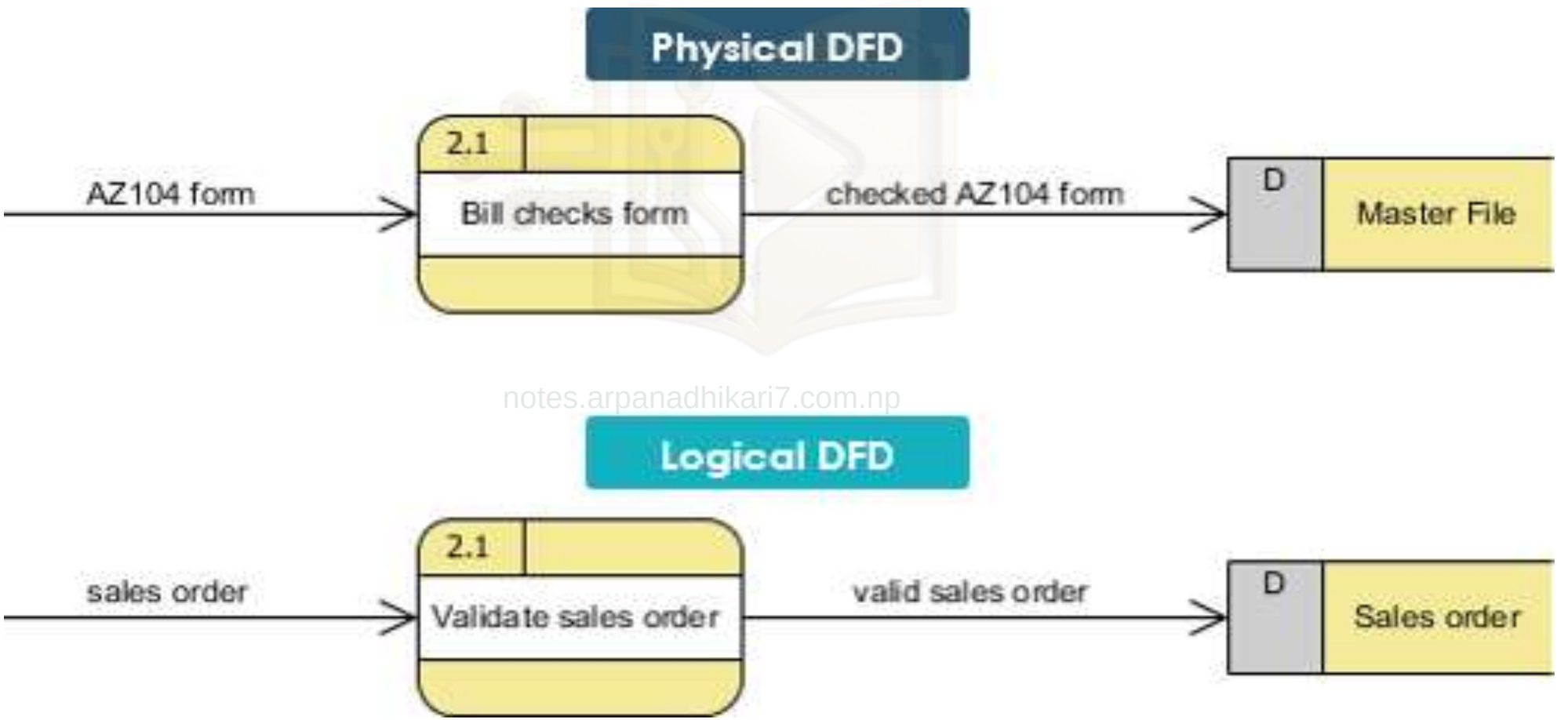
Logical DFD	Physical DFD
Logical DFD depicts how the business operates.	Physical DFD depicts how the system will be implemented (or how the current system operates).
The processes represent the business activities.	The processes represent the programs, program modules, and manual procedures.
The data stores represent the collection of data regardless of how the data are stored.	The data stores represent the physical files and databases, manual files.
It s how business controls.	It show controls for validating input data, for obtaining a record, for ensuring successful completion of a process, and for system security.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Major Differences between logical and physical dfds

1. Physical DFD specifies actual flow of physical documentation(entities, parameters), while logical DFD only focus on the information flow in business term.

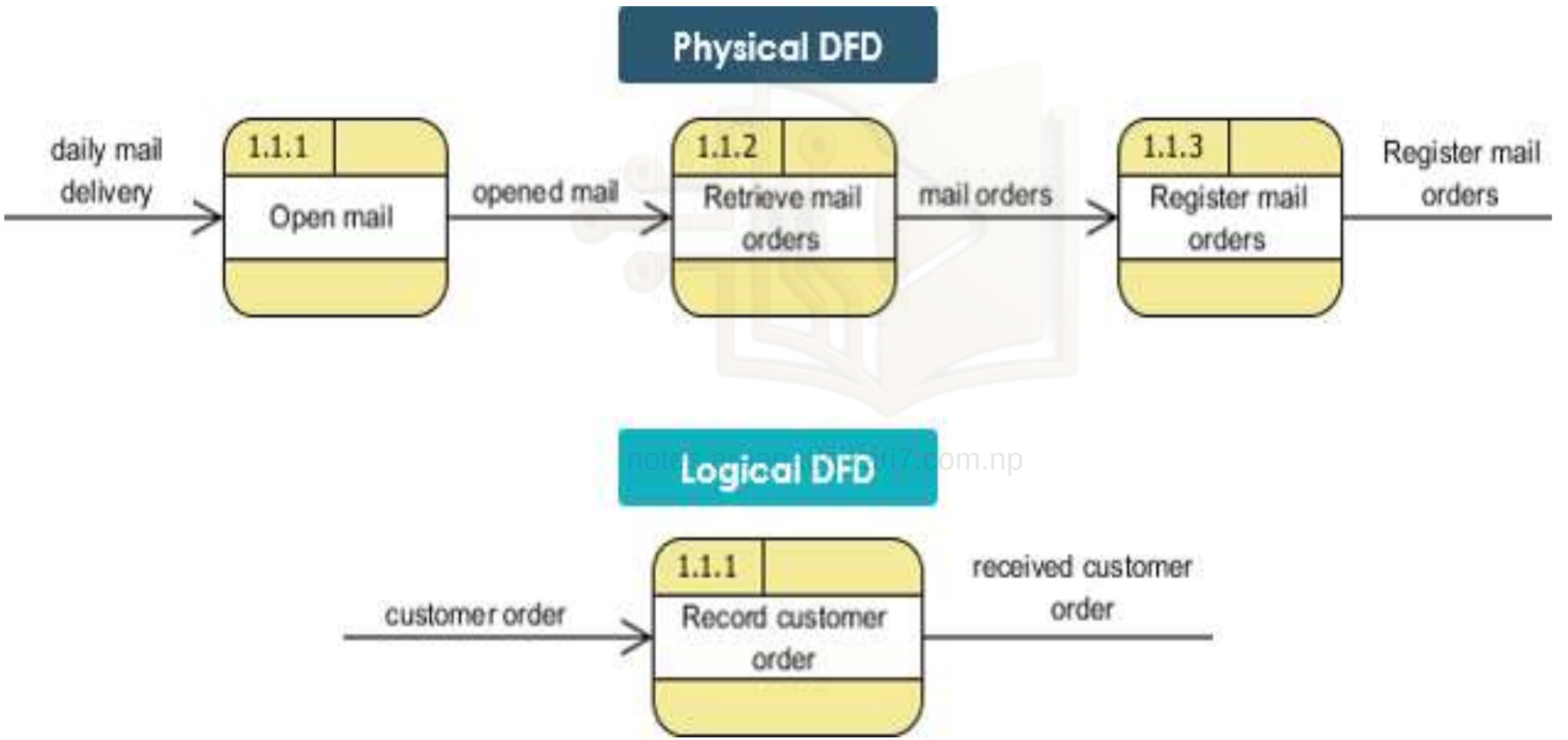


notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Major Differences between logical and physical dfds

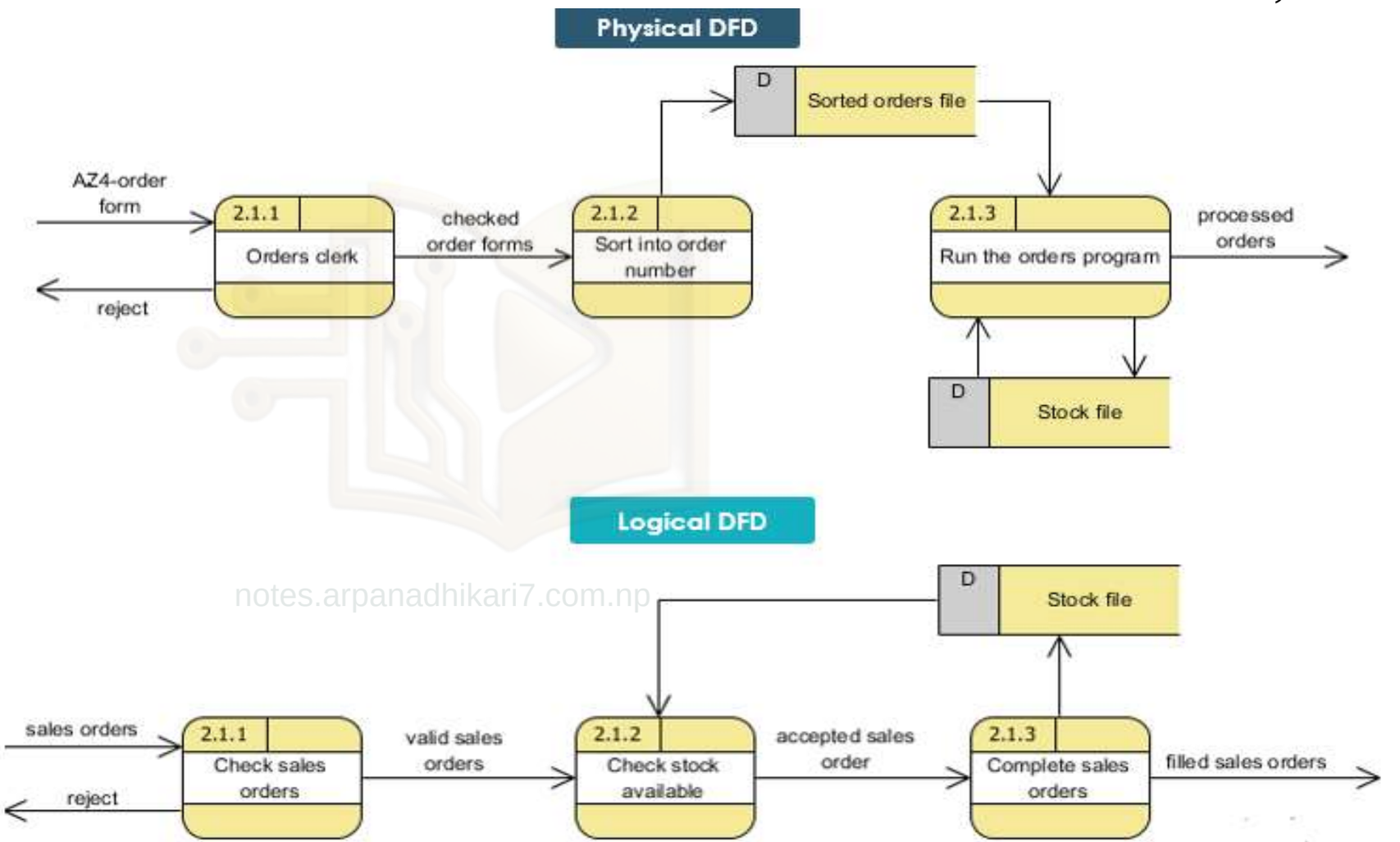
2. Logical DFD eliminates physical processes that refer to physical activities only and do not transform data.



Available on notes.arpanadhikari7.com.np

Major Differences between logical and physical dfds

3. The logical DFD describes what the system does by including the essential sequence of business activities. It model the business data and activities instead of actual forms, location and roles.



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

DFD Rules and Tips

1. Each process should have at least one input and an output.
2. Each data store should have at least one data flow in and one data flow out.
3. Data stored in a system must go through a process.
4. All processes in a DFD go to another process or a data store.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

DFD levels and layer

- A data flow diagram can dive into progressively more detail by using levels and layers.
- DFD levels are numbered 0, 1 or 2, and occasionally go to even Level 3 or beyond.
- The necessary level of detail depends on the scope of what we are trying to accomplish.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Top-Down Decomposition Techniques

- Top-down decomposition, also called leveling, is a technique used to show more detail in lower-level DFDs.
- Leveling is done by drawing a series of increasingly detailed diagrams until the desired degree of detail is reached.
- **Balancing DFD**
  - When performing top-down decomposition to a DFD to lower level DFDs, the inputs and outputs must be conserved between levels of DFDs.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

How to create a data flow diagram?

1. Identify major inputs and outputs in your system
2. Build a context diagram
   - Once we have identified the major inputs and outputs, building a context diagram is simple.
   - Draw a single process node and connect it to related external entities.
   - This node represents the most general process information undergoes to go from input to output.
3. Expand the context diagram into a level 1 DFD
   - The single process node of your context diagram doesn’t provide much information
   - So we need to break it down into subprocesses.
   - In our level 1 data flow diagram, we should include several process nodes, major databases, and all external entities.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

How to create a data flow diagram?

3. Expand to a level 2+ DFD
   - The processes in your level 1 DFD can be broken down into more specific subprocesses.
   - Once again, we add any necessary data stores and flows—at this point we should have a fairly detailed breakdown of our system.
   - To progress beyond a level 2 data flow diagram, simply repeat this process.
4. Confirm the accuracy of our final diagram
   - When our diagram is completely drawn, we walk through it.
   - We check whether all processes, necessary data stores included or not.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Level 0 DFD

- DFD Level 0 is also called a Context Diagram.
- It's a basic overview of the whole system or process being analyzed or modeled.
- It's designed to be an at-a-glance view, showing the system as a single high-level process, with its relationship to external entities.
- It should be easily understood by a wide audience, including stakeholders, business analysts, data analysts and developers.
- It contains one and only one process and does not show any data store.
- Rule:
  - Only one process
  - Data flows should be labeled.
  - No data store can be shown in context diagram

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

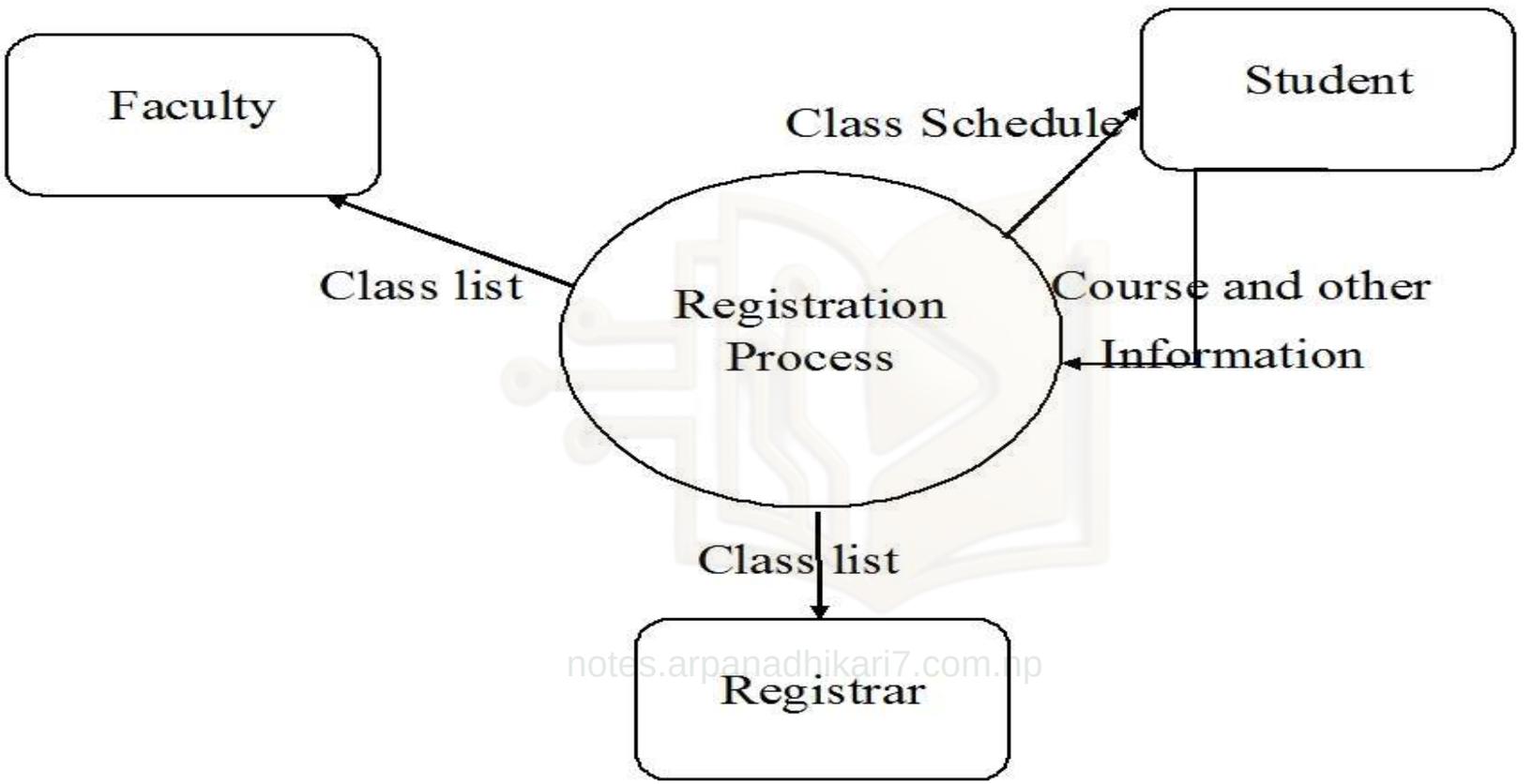


Figure: Level 0 DFD for Course Registration System

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Level 1 DFD

- As described previously, context diagrams (level 0 DFDs) are diagrams where the whole system is represented as a single process.
- DFD Level 1 provides a more detailed breakout of pieces of the Context Level Diagram.
- We will highlight the main functions carried out by the system, as we break down the high-level process of the Context Diagram into its subprocesses.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

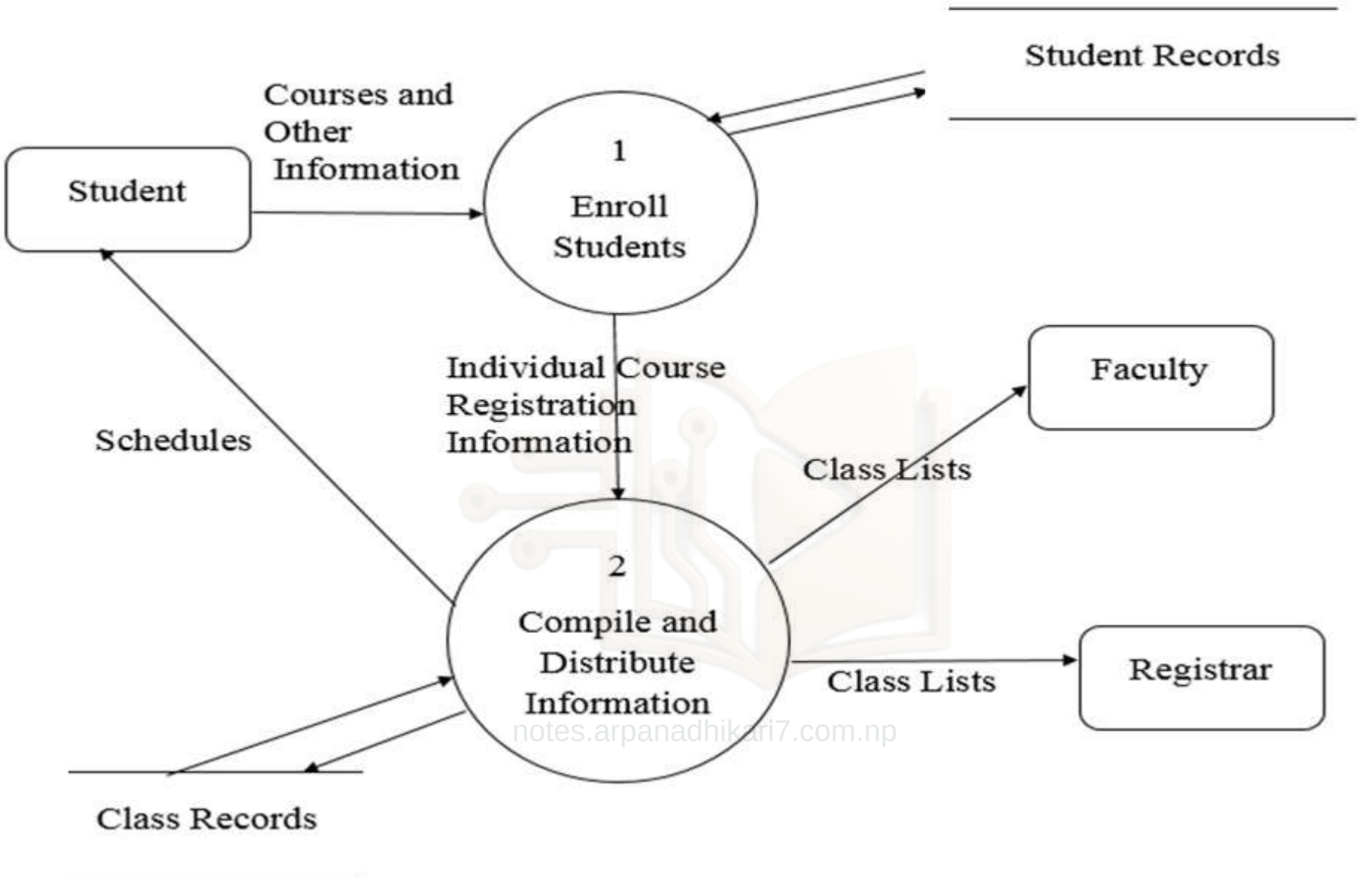


Figure: Level 1 DFD for Course Registration System

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

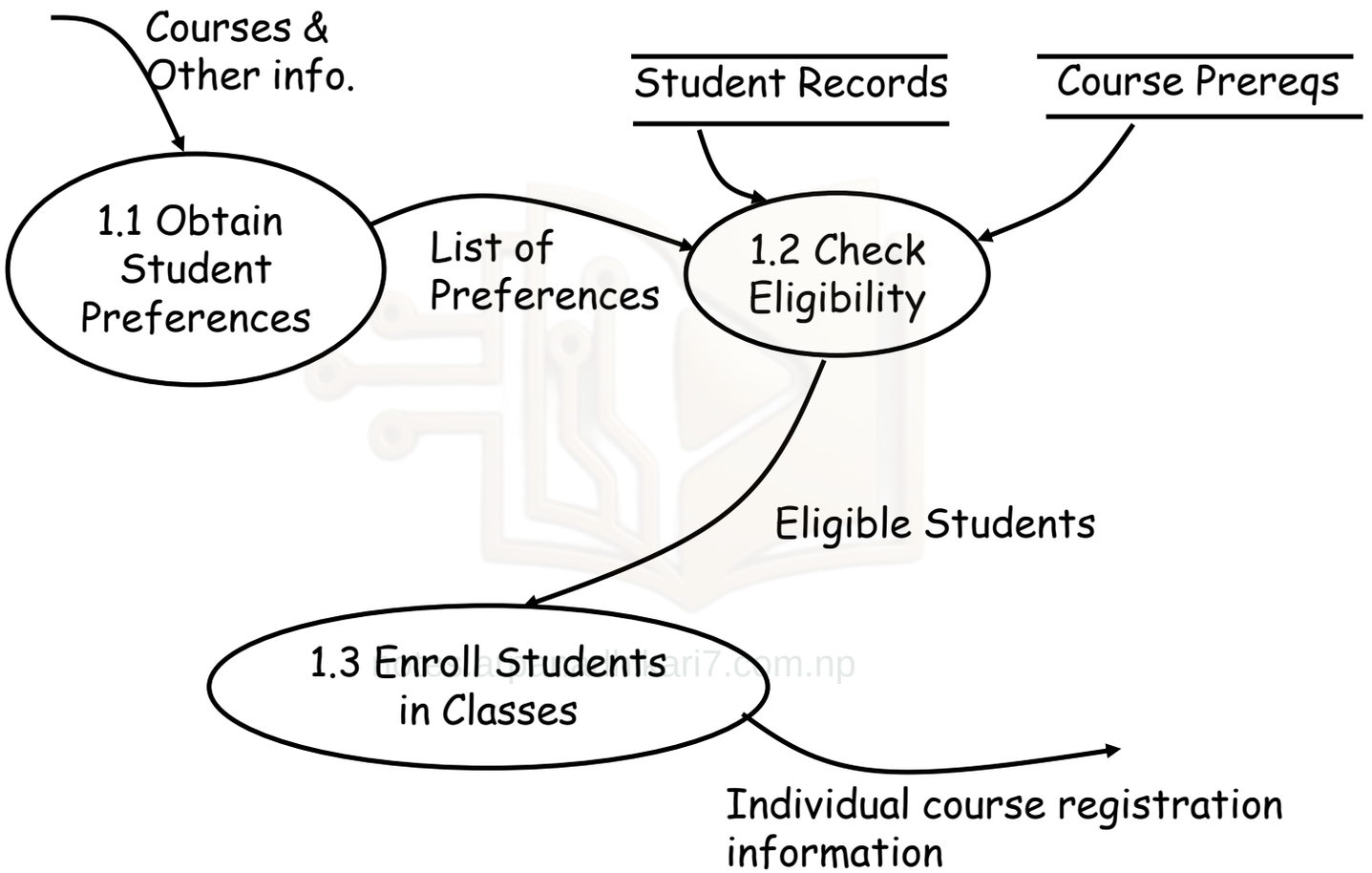
Level 2 DFD

- As described previously, context diagrams (level 0 DFDs) are diagrams where the whole system is represented as a single process.
- DFD Level 1 provides a more detailed breakout of pieces of the Context Level Diagram.
- We will highlight the main functions carried out by the system, as we break down the high-level process of the Context Diagram into its subprocesses.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Level 2 DFD for Course Registration System



Available on notes.arpanadhikari7.com.np

Data Flow Diagramming Mistakes

- One of the patterns of data flow analysis is that all flows must begin with or end at a processing step.
- Illegal Data Flows - Black Holes, Grey Holes and Miracles

  - A processing step may have input flows but no output flows. This situation is sometimes called a black hole.
  - A processing step may have output flows but now input flows. This situation is sometimes called a miracle.
  - A processing step may have outputs that are greater than the sum of its inputs This situation is sometimes referred to as a grey hole.

Available on notes.arpanadhikari7.com.np

UML(Unified Modeling Language)

- The Unified Modeling Language(UML) is a visual language for specifying, constructing and documenting the artifacts(work product) of systems.
- Unified Modeling language (UML) is a standardized modeling language enabling developers to specify, visualize, construct and document artifacts of a software system.
- UML is an important aspect involved in object-oriented software development. It uses graphic notation to create visual models of software systems.
- UML is not a system design or development methodology, but can be used to document object-oriented and analysis results obtained using some methodology.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

UML 1 and UML 2

- UML is being continually refined.
- UML 2 extends UML specifications to cover more aspects of development, including Agile.
- The goal was to restructure and refine UML so that usability, implementation, and adaptation are simplified.
- Some updates made from UML 1 to UML 2 are:

a. Greater integration between structural and behavior models.

b. Ability to define hierarchy and breakdown a software system into components and sub-components.

c. UML 2 raises the number of diagrams i.e. adds new types of diagrams.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Behavioral Models

- Behavioral models are UML diagrams focus on illustrating the dynamic aspects of a software system, showcasing how it behaves, responds to stimuli, and undergoes state changes during runtime.

notes.arpanadhikari7.com.np

Behavioral Model

- Behavioral model describes the interaction in the system.
- Behavioral modeling shows the dynamic nature of the system.
- Modeling techniques used in it are use-case diagram, activity diagram, sequence diagram, etc.
- The time and cost for developing a system using behavioral model is comparatively less compared to structural model.
- Diagram used in behavioral model are Activity Diagram, Use Case Diagram, Sequence Diagram, Communication Diagram, State Machine Diagram etc.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Interaction Models

- Interaction models shows the interaction between the components of a system, or between the system being developed and other systems.
- Modeling user interaction is important as it helps to identify user requirements
- Modeling system-to-system interaction highlights the communication problems that may arise.
- Use case diagrams, sequence diagrams, communication diagram may be used for interaction modeling.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Use Cases

- A use case is a list of actions or event steps typically defining the interactions between a role or actor and a system to achieve a goal.
- Use cases are not diagrams, they are text representing list of actions or events event steps.
- Use Case, is a name for a scenario to describe the user–computer system interaction.
- Use Cases are used to determine system requirements, identify classes and their relationship to other classes in domain.
- Requirements analysis may include stories or scenarios of how people use the application; these can be written as use cases. They are a popular tool in requirements analysis.
- Use cases can be used to examine who does what in interactions among objects, what role they play, intersection among objects' role to achieve given goal.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Use Case Modeling

- It is the process of expressing high level processes and interactions with customers in a scenario and analyzing it .It gives system uses, system responsibilities.
- Developing use case is iterative process. When use case model are better understood and developed, we then start identifying classes and create their relationship.
- Use case modeling helps in:
    a. Capturing requirements
    b. Planning iterations of development
    c. Validating systems
    d. Expressing high level processes and interactions with customers in a scenario and analyzing it.
    e. Providing system uses, system responsibilities.

Available on notes.arpanadhikari7.com.np

Scenarios	Use Case
It is a great way to establish communication with client. It is an instance of use case.	Use Case is an abstraction of scenarios which shows the interactions between an actor and system.
The focus is on understandability.	The focus is on completeness.

Format of Use Case

a. **Brief Use Case:** One-paragraph summary, usually of the main success scenario.
b. **Causal Use Case:** Informal paragraph format. Multiple paragraphs that cover various scenarios.

- **Fully dressed Use Case : It is** the most elaborated use case format. All steps and variations are written in detail, and there are supporting sections, such as preconditions and success guarantees. Fully Dressed Use Cases show more detail and are structured. They dig deeper. Fully dressed use case is widely used and shared format. Template is as below:

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

➢A use case basically describes the following three things:

a. An actor(user) that initiates.

b. An event that triggers a use case.

c. The use case that performs the actions triggered by the use case.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Use Case Diagram

- Use case diagrams model the functionality of a system using actors and use cases.
- A use case diagram is a graphic depiction of the interactions among the elements of a system.
- Use case diagrams are used to depict the context of the system to be built and the functionality provided by that system.
- They show what the outside world wants the system to do.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Why make use case diagrams?

- Use case diagrams are valuable for visualizing the functional requirements of a system that will translate into design choices and development priorities.
- Use case diagrams are used to gather the requirements of a system including internal and external influences. They help to identify the external and internal factors influencing system.
- Use case diagrams can be a good communication tool for stakeholders. The drawings attempt to mimic the real world and provide a view for the stakeholder to understand how the system is going to be designed.
- They help to show the interaction and relationship among the use cases and requirements.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Major Elements of Use Case Diagram

1. Actor

- An actor can be defined as some internal or external entity that interacts with the system.
- Actors are not only roles played by people, but organizations, software, and machines.
- An Actor models a type of role played by an entity that interacts with the subject (e.g., by exchanging signals and data), but which is external to the subject.
- Basically there are two types of actors in a system:

a. Primary Actor: Actor using the system to achieve a goal. Initiates the use of the system.

b. Secondary Actor: Actor that the system needs assistance from to achieve primary actor's goal.

notes.arpanadhikari7.com.np

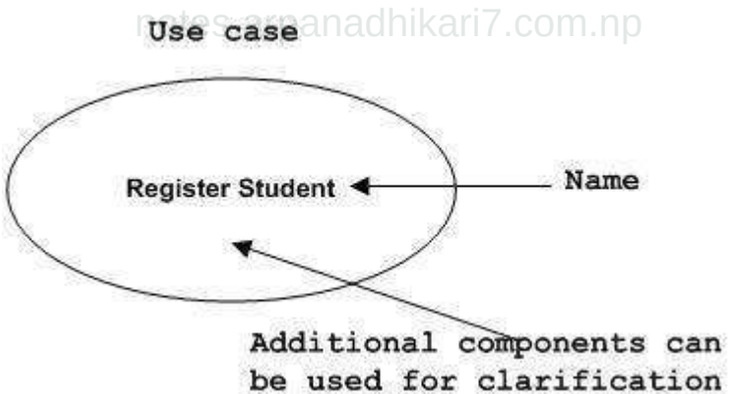


Available on notes.arpanadhikari7.com.np

2. Use Cases

- Use cases represent what the actors want your system to do for them.
- A good way of identifying use cases is to recognize what the actor needs from the system or what the actor can do on the system.
- The use case is made up of a set of possible sequences of interactions between systems and users in a particular environment and related to a particular goal.
- Use case **start with verb** and reinforces action that take place.
- In a banking system, a customer will need to open accounts, deposit and withdraw funds, request check books and similar functions. So all of these can be considered as use cases.
- Top level use cases should always provide a complete function required by an actor. You can **extend** or **include** use cases depending on the complexity of the system.

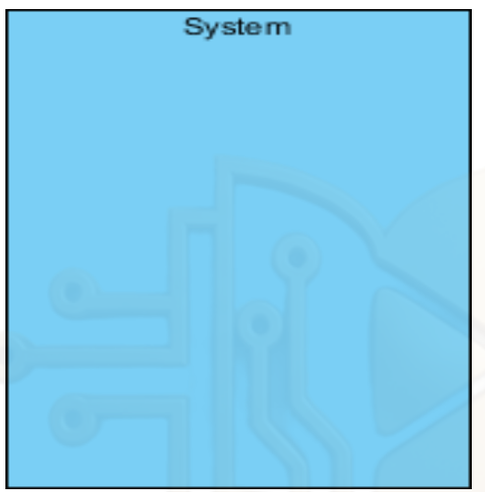
notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

3. System Boundary

- To represent the system a rectangle or rectangular box is used.
- We can use system boundary to group logically related elements.



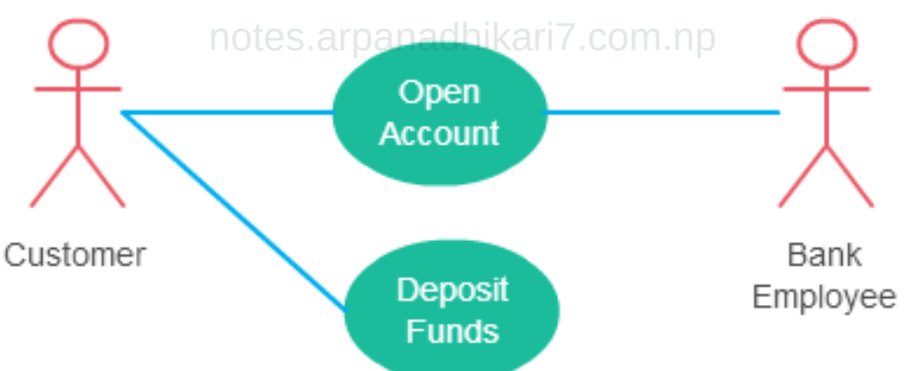
notes.arpanadhikari7.com.np

Relationships in use case diagram

1. Association Between Actor and Use Case

- An association relationship is a structural relationship between two model elements that shows that objects of one classifier (actor, use case, class, interface, node, or component) connect and can navigate to objects of another classifier.
- Association is represented by a solid line.
- Things to remember while using association:

- An actor must be associated with at least one use case.
- An actor can be associated with multiple use cases.
- Multiple actors can be associated with a single use case.

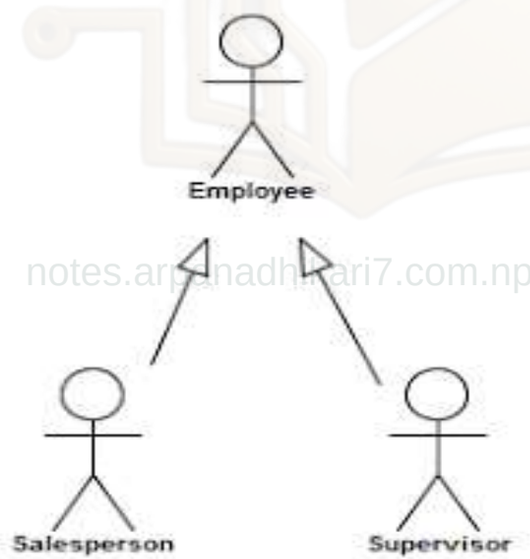


notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

2. Generalization of an actor

- Generalization is communication between an actor and a use case is represented by a solid line.
- Actor generalization refers to the relationship which can exist between two actors and which shows that one actor (descendant) inherits the role and properties of another actor (ancestor).
- The generalization relationship also implies that the descendant actor can use all the use cases that have been defined for its ancestor. i.e. the descendant inherits all the use cases of the ancestor.

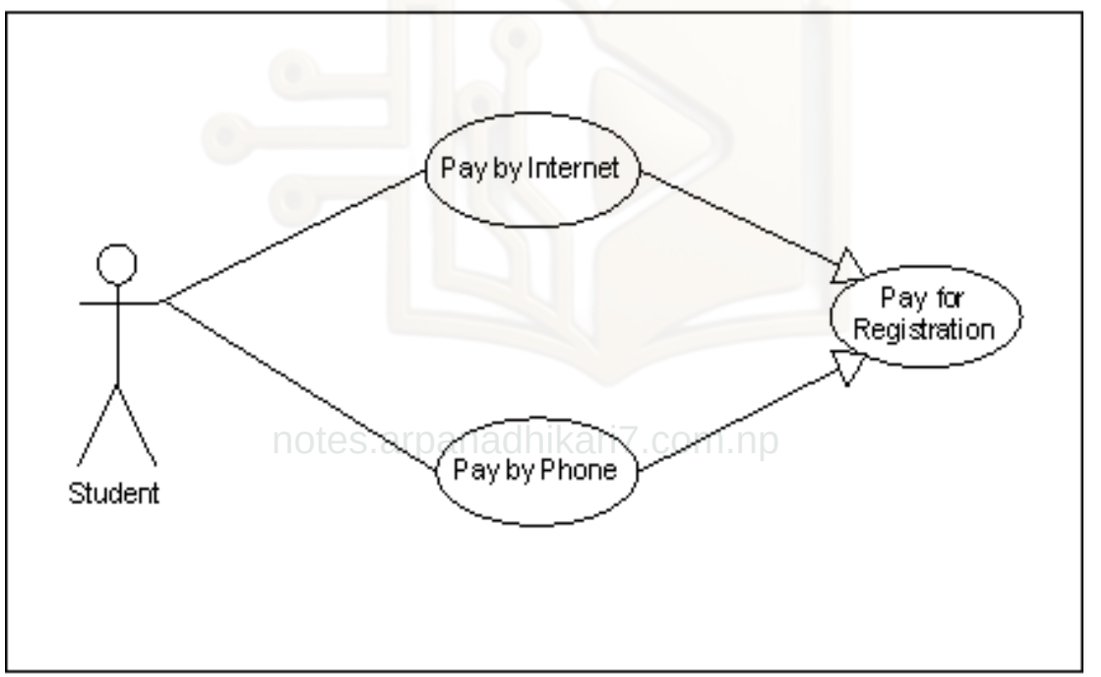


notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

3. Generalization of Use Case

- In generalization of use case, the behavior of the ancestor use case is inherited by the descendant use case.
- We can use the generalization relationship when we find two or more use cases which have common behavior/logic.



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

4. Include Relationship

- Include is a relationship between two use cases which is used to show that behavior of the included use case is inserted into the behavior of the including (the base) use case.
- Use case include is a directed relationship between two use cases which is used to show that behavior of the included use case is inserted into the behavior of the including (the base) use case.
- Every time the base use case is executed, the included use case is executed as well.
- To show include relationship, an arrow line from base to included use case.
- The **include** relationship could be used to:

a. simplify large use case by splitting it into several use cases

b. extract common parts of the behaviors of two or more use cases

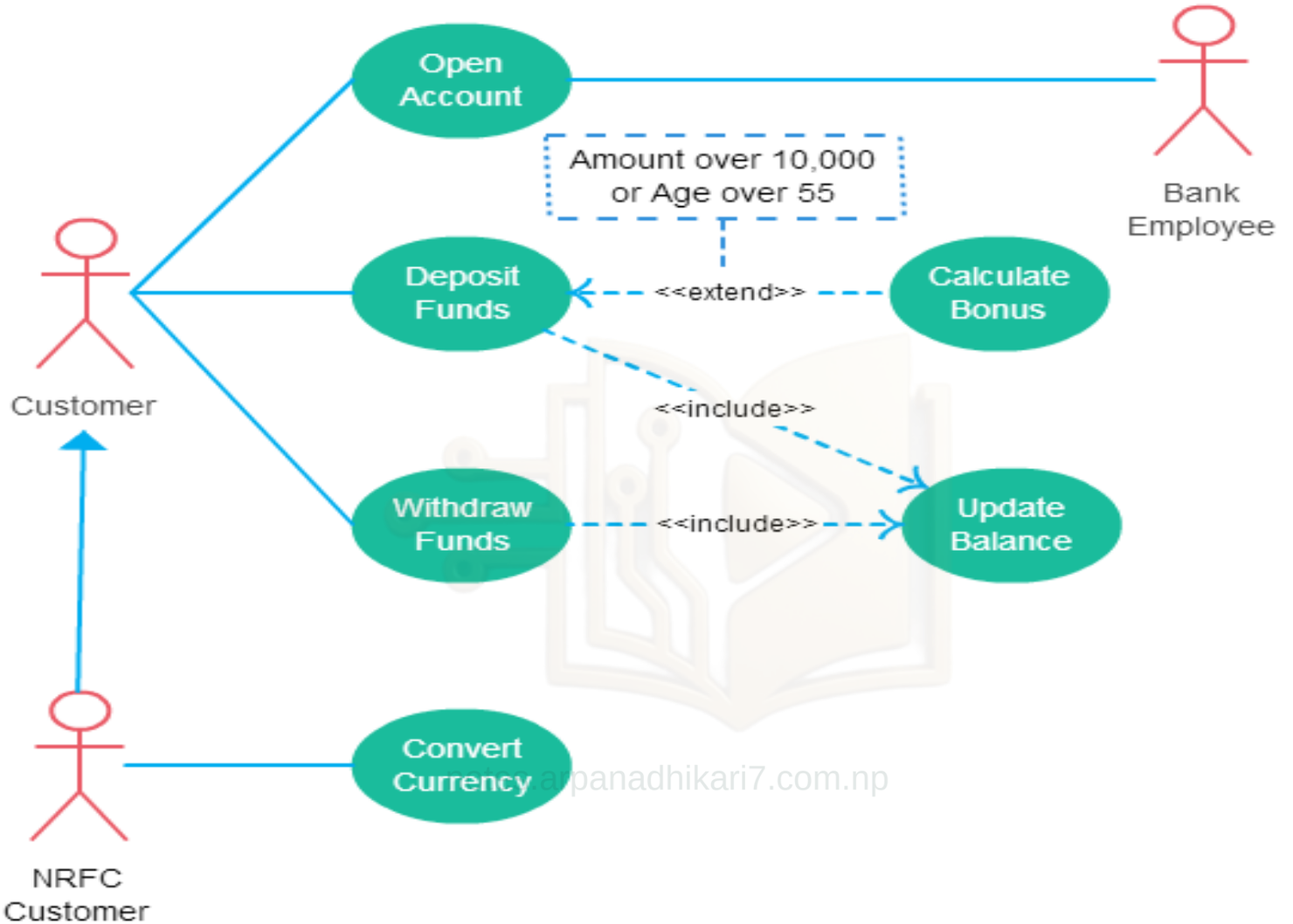
Available on notes.arpanadhikari7.com.np

5. Extend Relationship

- Extend is a directed relationship that specifies how and when the behavior defined in extending use case can be inserted into the behavior defined in the extended(base) use case.
- Extending use case typically defines **optional behavior** that is not necessarily meaningful by itself.
- There are some functions that are triggered optionally. In such cases, we can use the extend relationship.
- When the base use case is executed, the extending use case is executed only after meeting certain criteria.
- To show extend relationship, an arrow line from extending to base use case is used.

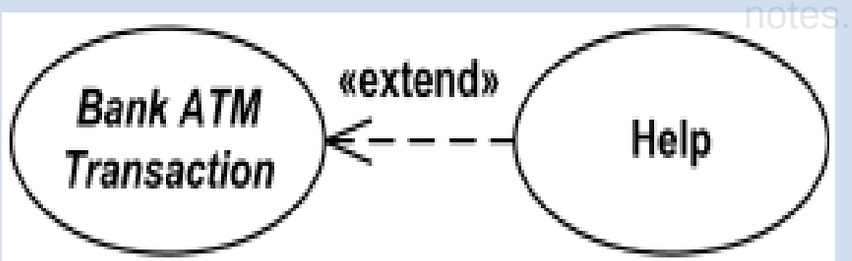
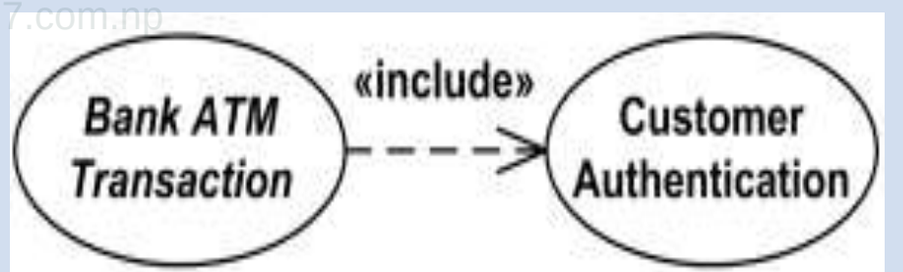
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

Extend vs Include Relationship

Extend	Include
Extend is a directed relationship that specifies how and when the behavior defined in extending use case can be inserted into the behavior defined in the extended(base) use case.	Include is a relationship between two use cases which is used to show that behavior of the included use case is inserted into the behavior of the including (the base) use case.
In extend relationship, base use case is complete on its own.	In include relationship, included use case is required.
Extending use case is optional.	Included use case is required.
To show extend relationship, an arrow line from extending to base use case is used.	To show include relationship, an arrow line from base to included use case.
	

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

How to draw a use case diagram?

1. Identifying Actor: Actors are external entities that interact with your system. It can be a person, another system or an organization.

Such actors are identified in this phase. In a banking system, the most obvious actor is the customer. Other actors can be bank employee or cashier depending on the role.

Primary Actor: Actor using the system to achieve a goal. Initiates the use of the system.

Secondary Actor: Actor that the system needs assistance from to achieve primary actor's goal.

2. Identifying Use Case: In this phase the use cases needed are to be identified. A good way to do this is to identify what the actor needs from the system or what the actor can do on the system.

notes.arpanadhikari7.com.np

Use case **start with verb** and reinforces action that take place.

In a banking system, a customer will need to open accounts, deposit and withdraw funds, request check books and similar functions. So all of these can be considered as use cases.

Top level use cases should always provide a complete function required by an actor. You can extend or include use cases depending on the complexity of the system.

Available on notes.arpanadhikari7.com.np

3. Look for Common Functionality to use include relationship: Include is a relationship between two use cases which is used to show that behavior of the included use case (the addition) is inserted into the behavior of the including (the base) use case.

It is a relationship in which one use case includes the functionality of another use case.

In case of include relationship, generally it exists between a base use case and included use case. Every time the base use case is executed, the included use case is executed as well.

To show relationship: Arrow line from base to included use case.

4. Is it Possible to Generalize Actors and Use Cases: We need to check whether the actors and use cases can be generalized or not. We can generalize the actor and use cases to show the inheritance of functions.

One of the best examples of this is "Make Payment" use case in a payment system. You can further generalize it to "Pay by Credit Card", "Pay by Cash", "Pay by Check" etc. All of them have the attributes and the functionality of a payment with special scenarios unique to them.

Available on notes.arpanadhikari7.com.np

5. Optional Functions or Additional Functions: There are some functions that are triggered optionally. In such cases, we can use the extend relationship.

These sorts of function only gets triggered when specified condition is matched.

In UML modeling, we can use an **extend relationship** to specify that one use case extends the behavior of another. This type of relationship reveals details about a system or application that are typically hidden in a use case.

An extend relationship has a base use case and extending use case. When the base use case is executed, the extending use case is executed only after meeting certain criteria.

To show relationship: Arrow line from extending to base use case.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

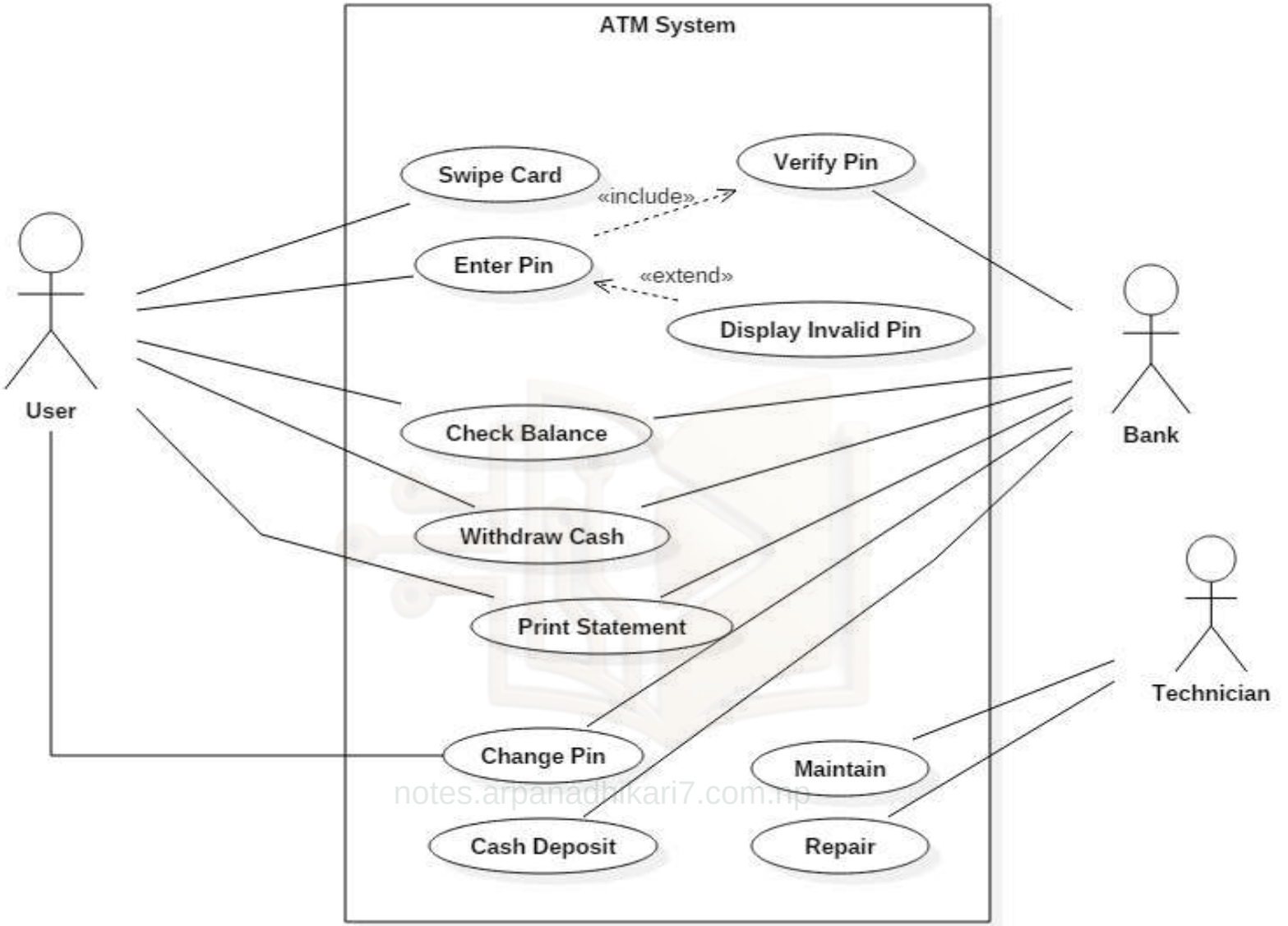


Figure: Use Case Diagram For ATM System

Available on notes.arpanadhikari7.com.np

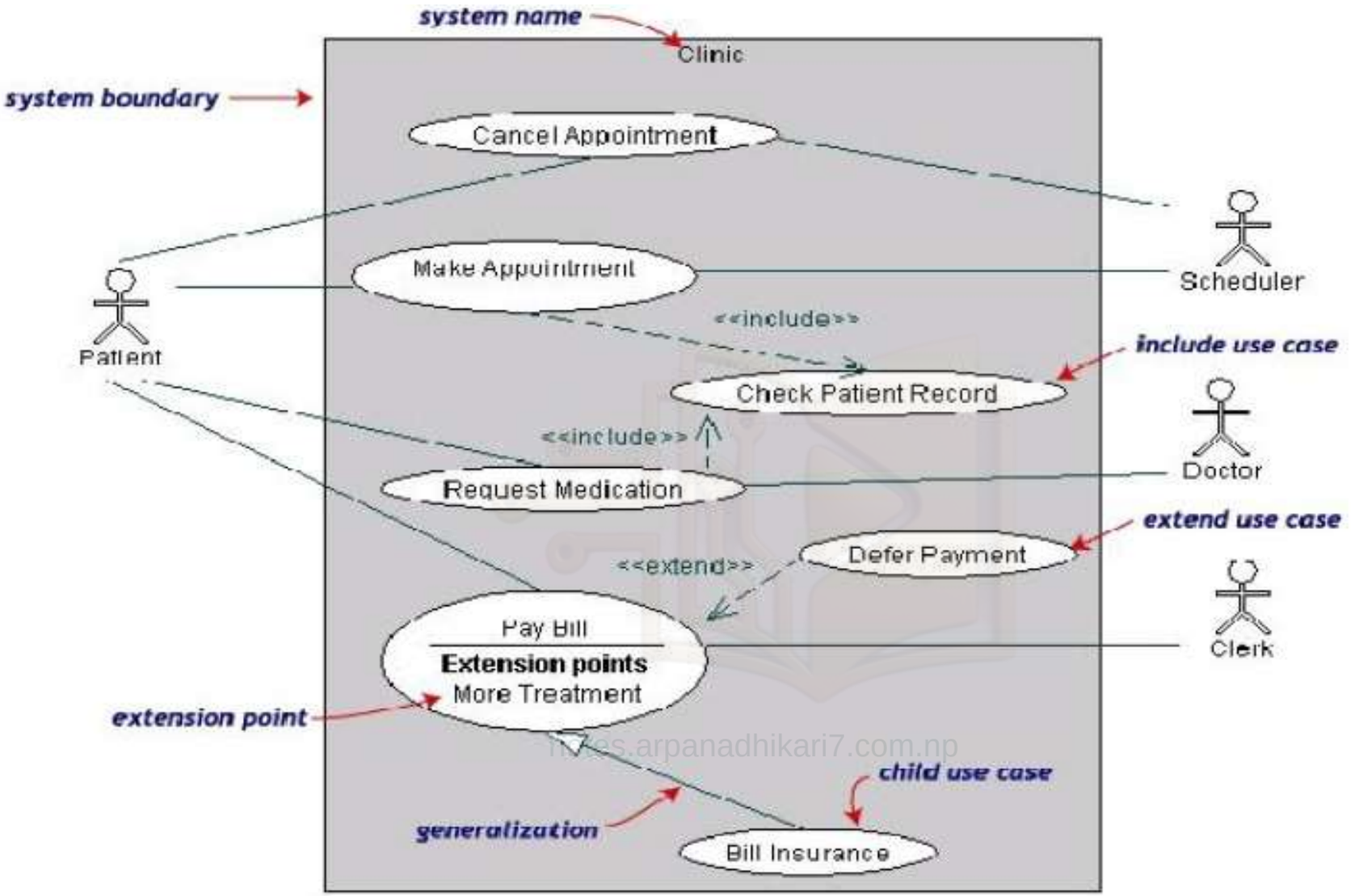


Figure: Use Case Diagram For Clinic System

Available on notes.arpanadhikari7.com.np

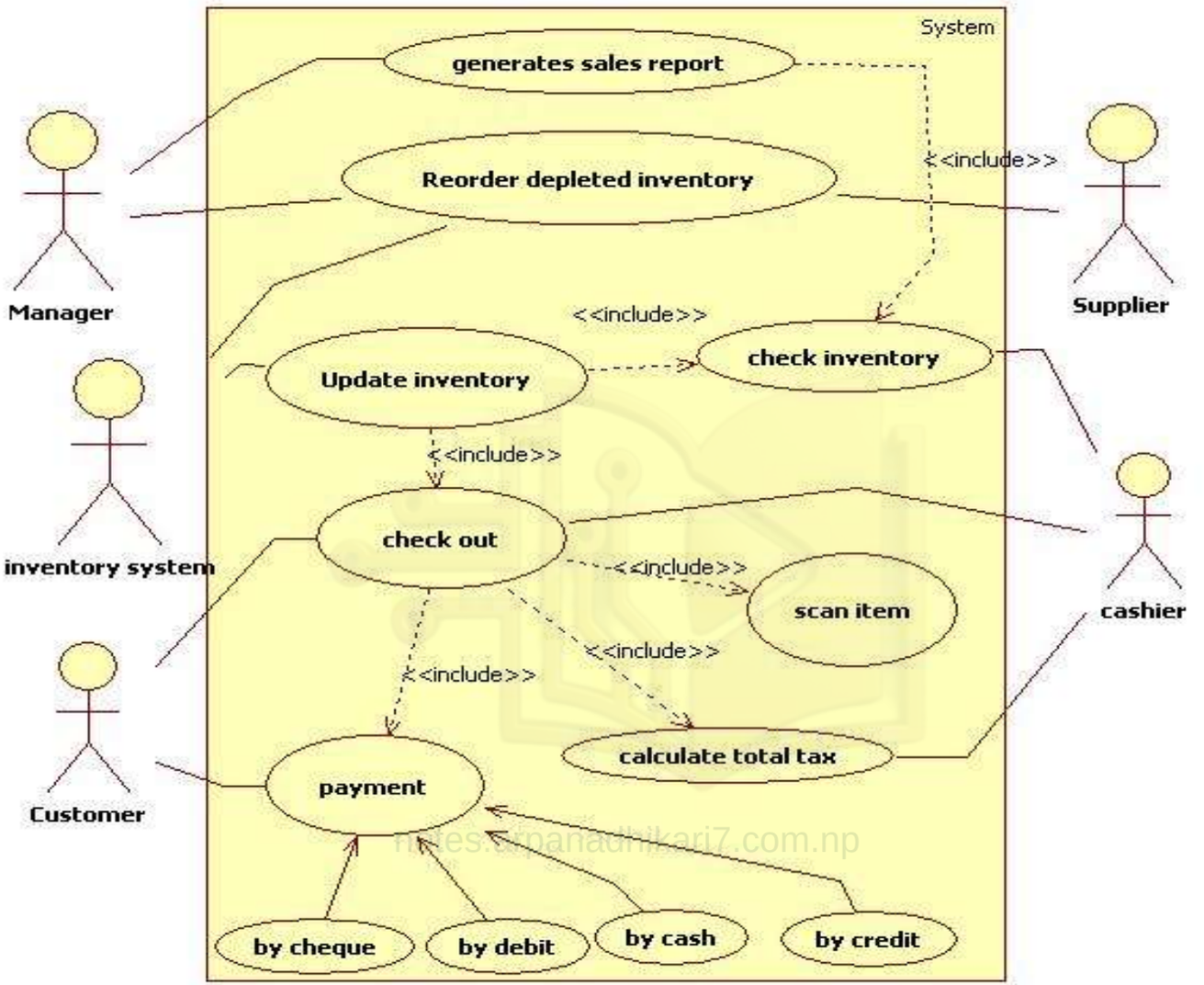


Figure: Use case diagram for POS

Available on notes.arpanadhikari7.com.np

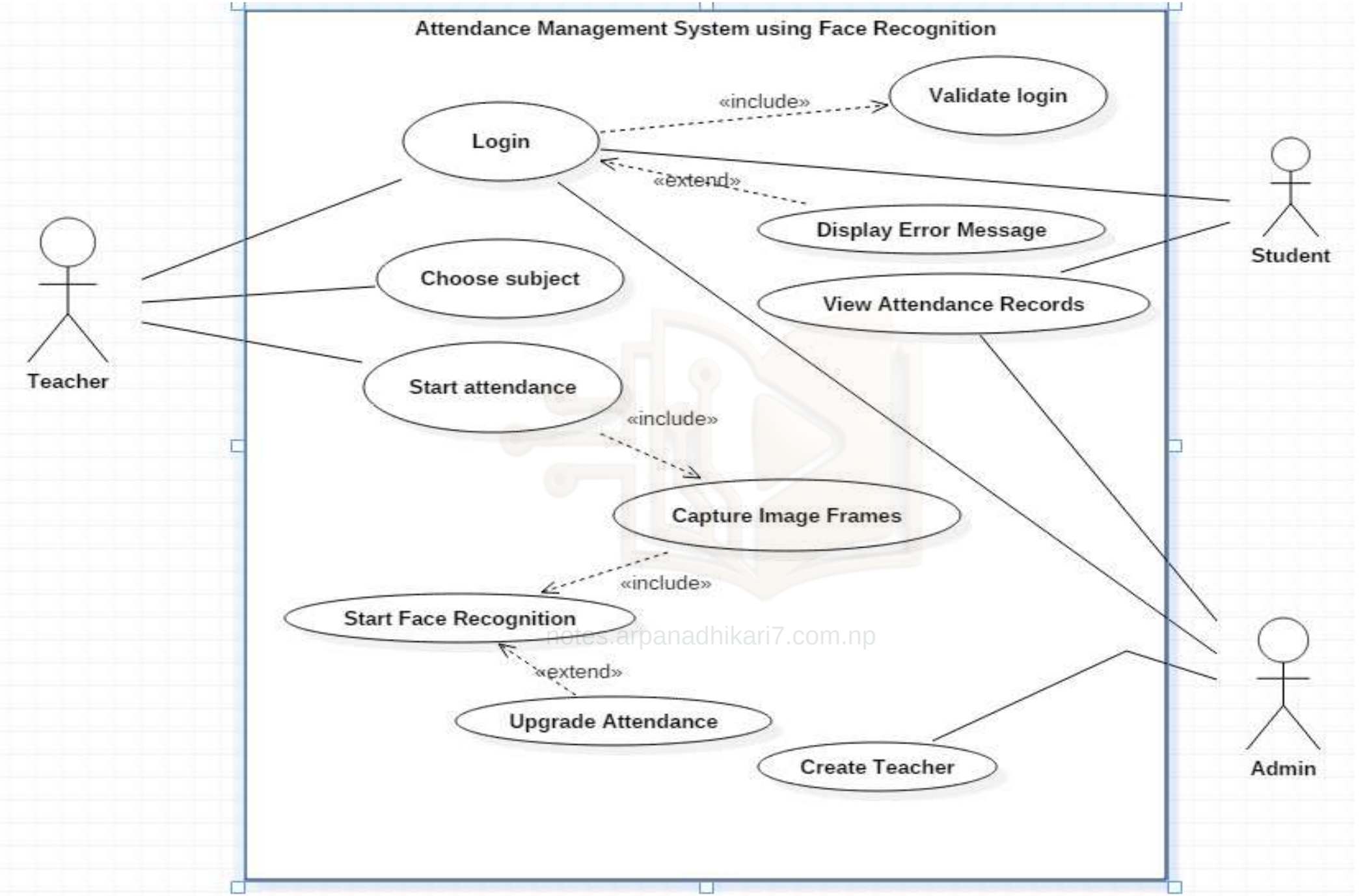
Example

Consider an attendance management system using face recognition for college students. Admin of the system creates teacher and can view attendance records. Each teacher must login to the system by selecting his/her subject as assigned by the admin. The teacher can start taking attendance of students . Provided image capturing device starts capturing image frame and recognizing face. If recognized, attendance is updated. Also students can login and view their attendance record.

Create a usecase diagram for above system with proper recognition of actors, usecases and relationships.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

System Sequence Diagram(SSD)

- A system sequence diagram is a picture that shows, for one particular scenario of a use case, the events that external actors generate, their order, and inter-system events.
- An SSD shows, for a particular course of events within a use case, the external actors that interact directly with the system, the system (as a black box), and the system events that the actors generate.
- The system sequence diagram is a tool for achieving more formal and detailed use case descriptions.
- All systems are treated as a black box.
- The emphasis of the diagram is events that cross the system boundary from actors to systems.

notes.arpanadhikari7.com.np

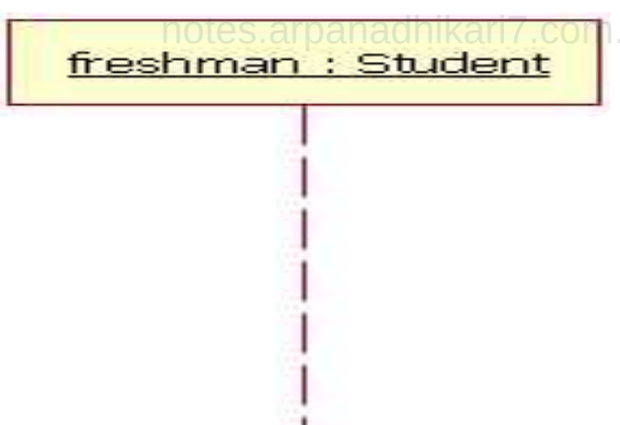
Available on notes.arpanadhikari7.com.np

Basic notations and Ideas for Sequence Diagram

a. Lifeline Notation

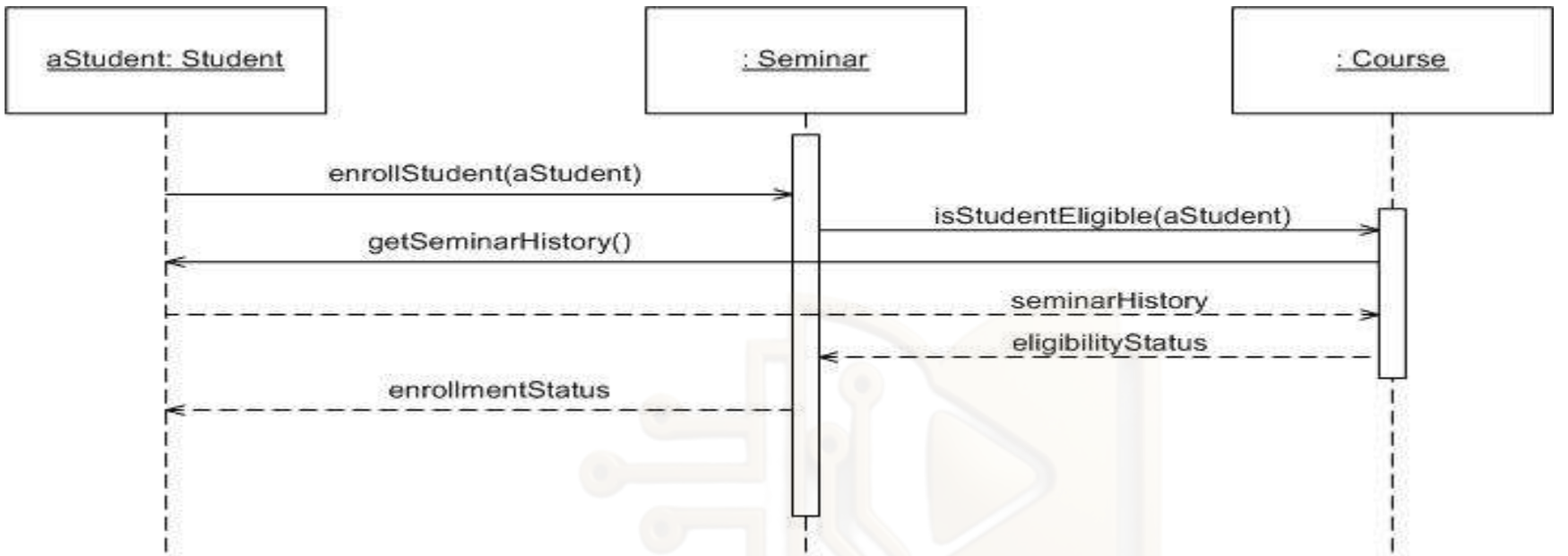
- In sequence diagram, lifelines represent the **objects or instances that participate in an interaction.**
- A sequence diagram is made up of several of these lifeline notations that should be arranged horizontally across the top of the diagram.
- No two lifeline notations should overlap each other.
- **Lifeline element notation with an actor element** is represented as:





notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



- In above figure, we see the Student object has the name aStudent, this is called a named object, whereas the instance of Seminar is an anonymous object.
- The instance of Student was given a name because it is used in several places as a parameter in messages, whereas the instance of the Seminar didn't need to be referenced anywhere else in the diagram and thus could be anonymous.

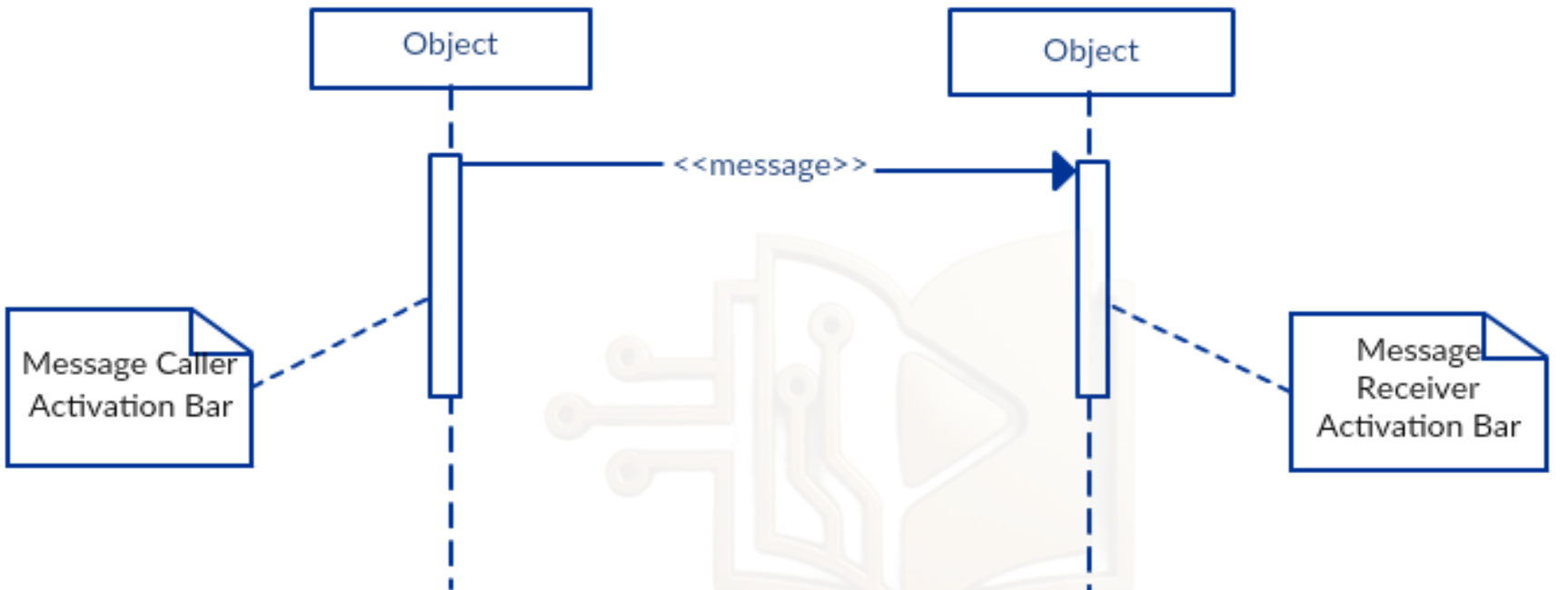
Available on notes.arpanadhikari7.com.np

b. Activation Bars

- Activation bar is the box placed on the lifeline.
- It is used to indicate that an object is active (or instantiated) during an interaction between two objects.
- The length of the rectangle indicates the duration of the objects staying active.
- In a sequence diagram, an interaction between two objects occurs when one object sends a message to another.
- The use of the activation bar on the lifelines of the Message Caller (the object that sends the message) and the Message Receiver (the object that receives the message) indicates that both are active during the exchange of the message.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

c. Destroy Object Notation or Destruction Occurrence

- Destruction occurrence is a message occurrence which represents the destruction of the instance described by the lifeline.
- It may result in the subsequent destruction of other objects that this object owns by **composition**.
- The destruction of instance is depicted by a cross in the form of an **X** at the bottom of a lifeline.



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

d. Message Arrows

- An arrow from the Message Caller to the Message Receiver specifies a message in a sequence diagram.
- A message can flow in any direction; from left to right, right to left or back to the Message Caller itself.
- The message arrow comes with a description, which is known as a message signature, on it.
- The format for this message signature is below. All parts except the message_name are optional.

attribute = message_name (arguments): return_type

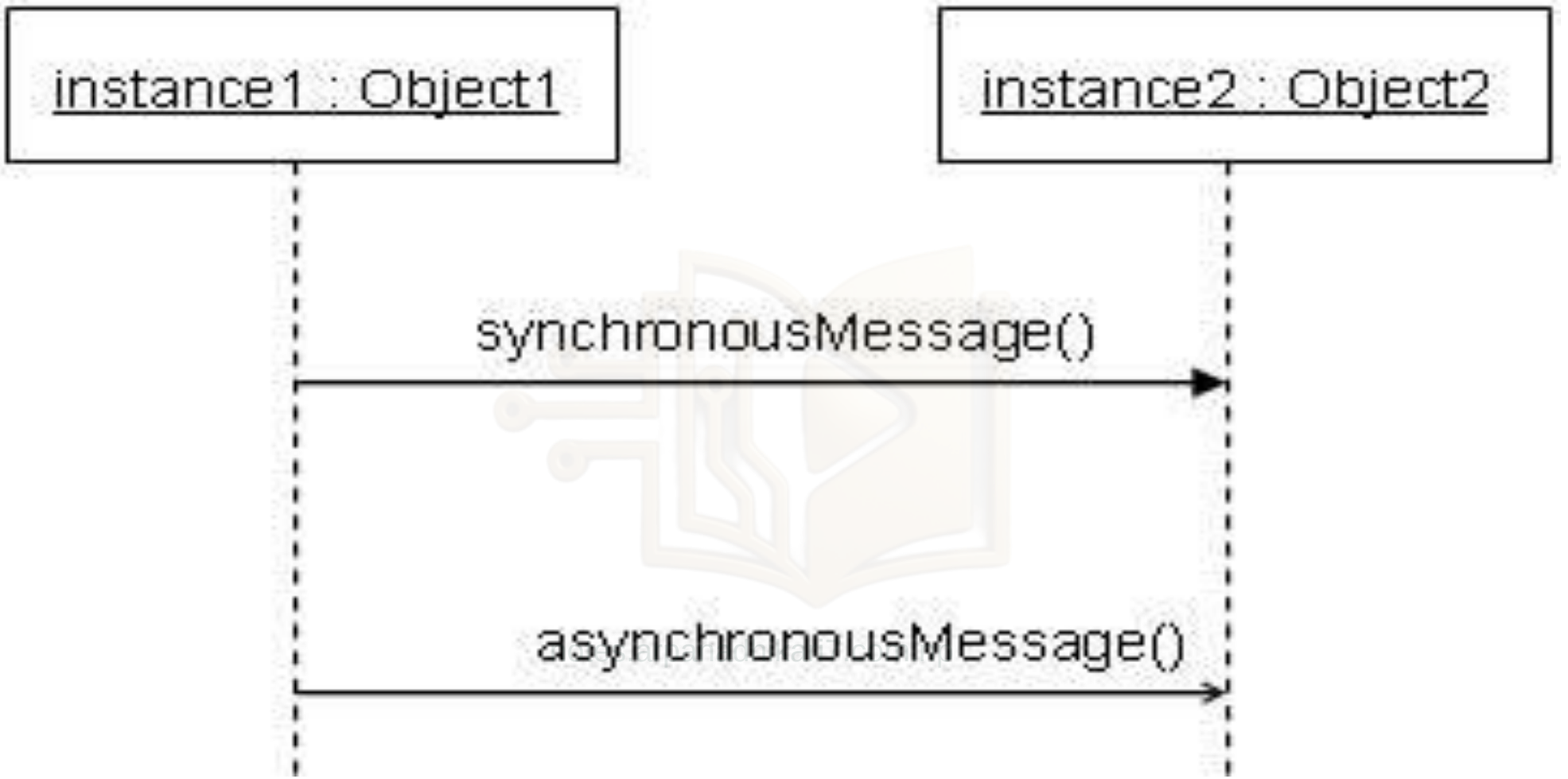
notes.arpanadhikari7.com.np

1. Synchronous message: A synchronous message is used when the sender waits for the receiver to process the message and return before carrying on with another message. The arrowhead used to indicate this type of message is a **solid one**.

2. Asynchronous message: An asynchronous message is used when the message caller does not wait for the receiver to process the message and return before sending other messages to other objects within the system. The arrowhead used to show this type of message is a **open line arrow**.

notes.arpanadhikari7.com.np

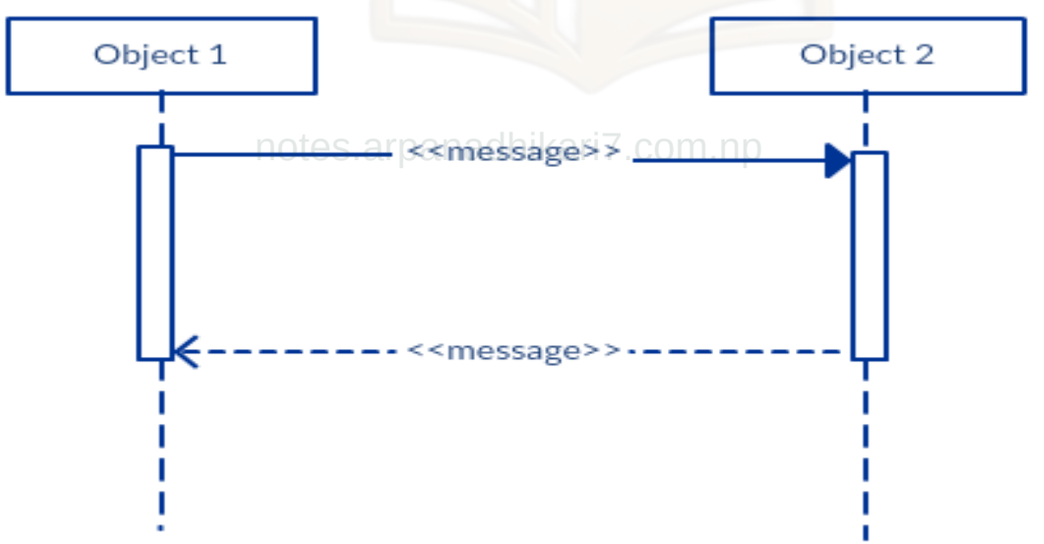
Available on notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

3. Return or Reply message

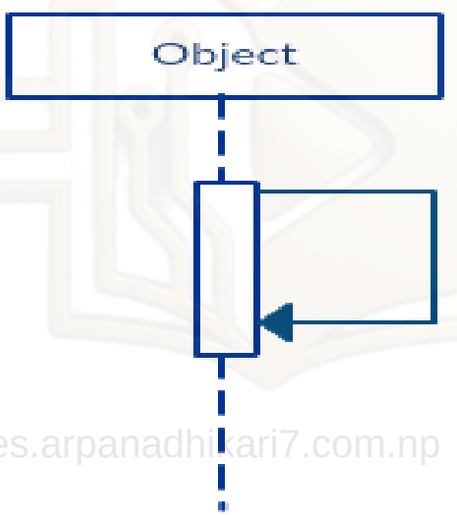
- A return message is drawn as a dotted line with an open arrowhead back to the originating lifeline.
- A return message is used to indicate that the message receiver is done processing the message and is returning control over to the message caller is used **as a reply to the earlier message.**
- These return messages are optional. The use of return messages depends on the level of detail or abstraction that is being modeled.



Available on notes.arpanadhikari7.com.np

4. Reflexive message

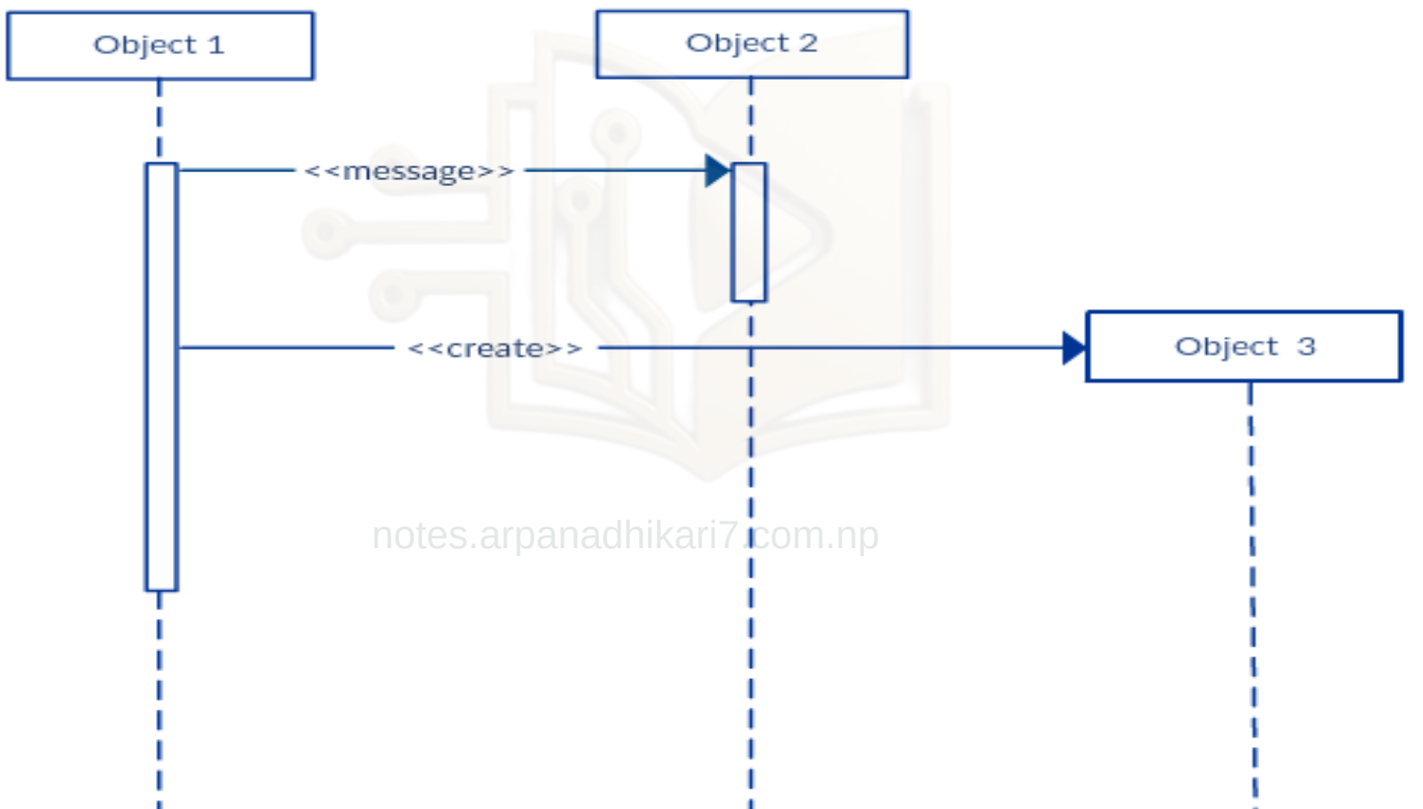
- When an object sends a message to itself, it is called a reflexive message.
- It is indicated with a message arrow that starts and ends at the same lifeline as shown in the example below.



notes.arpanadhikari7.com.np

5. Participant Creation Message

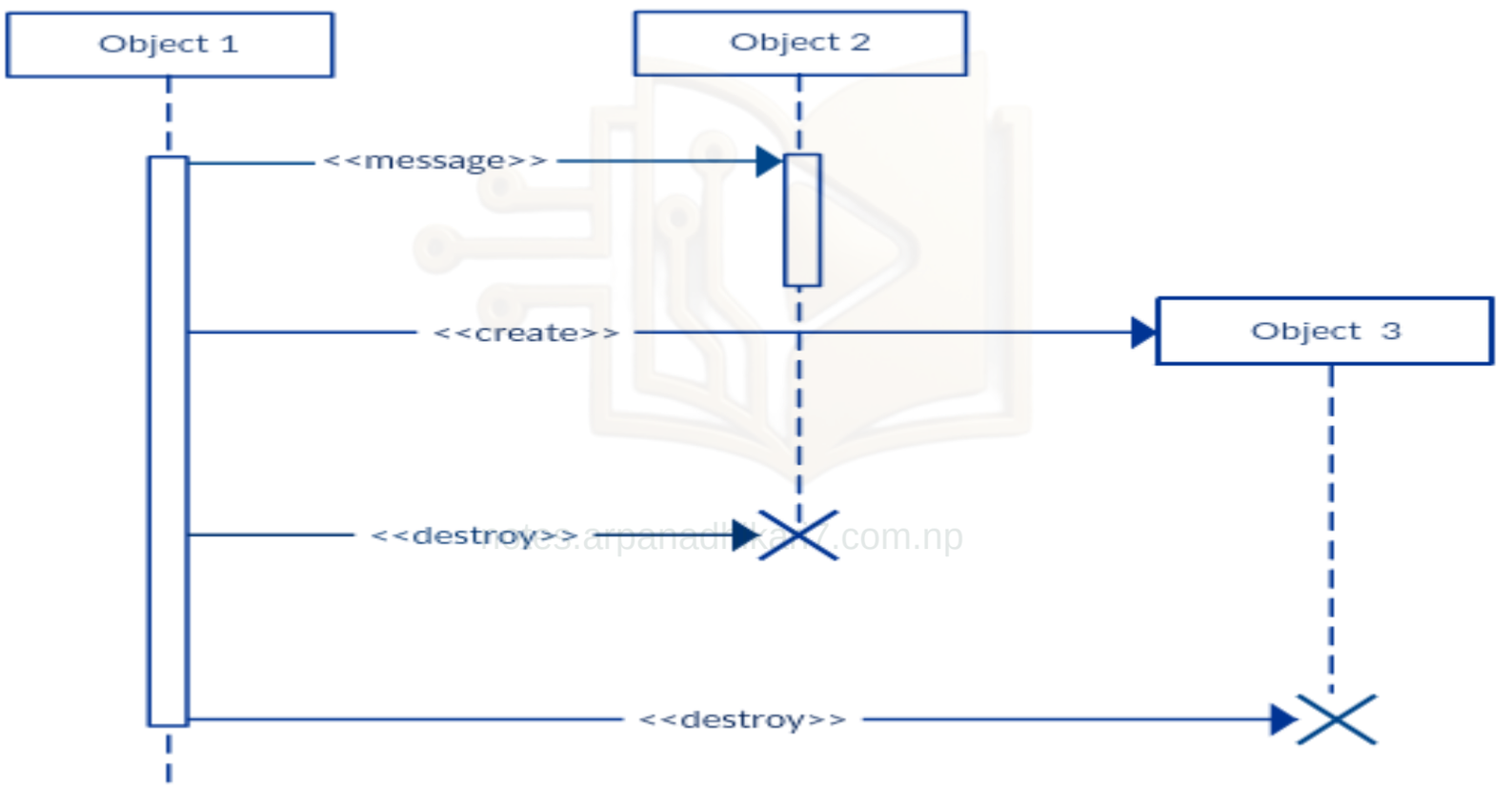
- Objects or participants can be created according to the message that is being sent.
- Objects or Participants do not necessarily live for the entire duration of the sequence of events.



notes.arpanadhikari7.com.np

6. Participant or Object destruction message

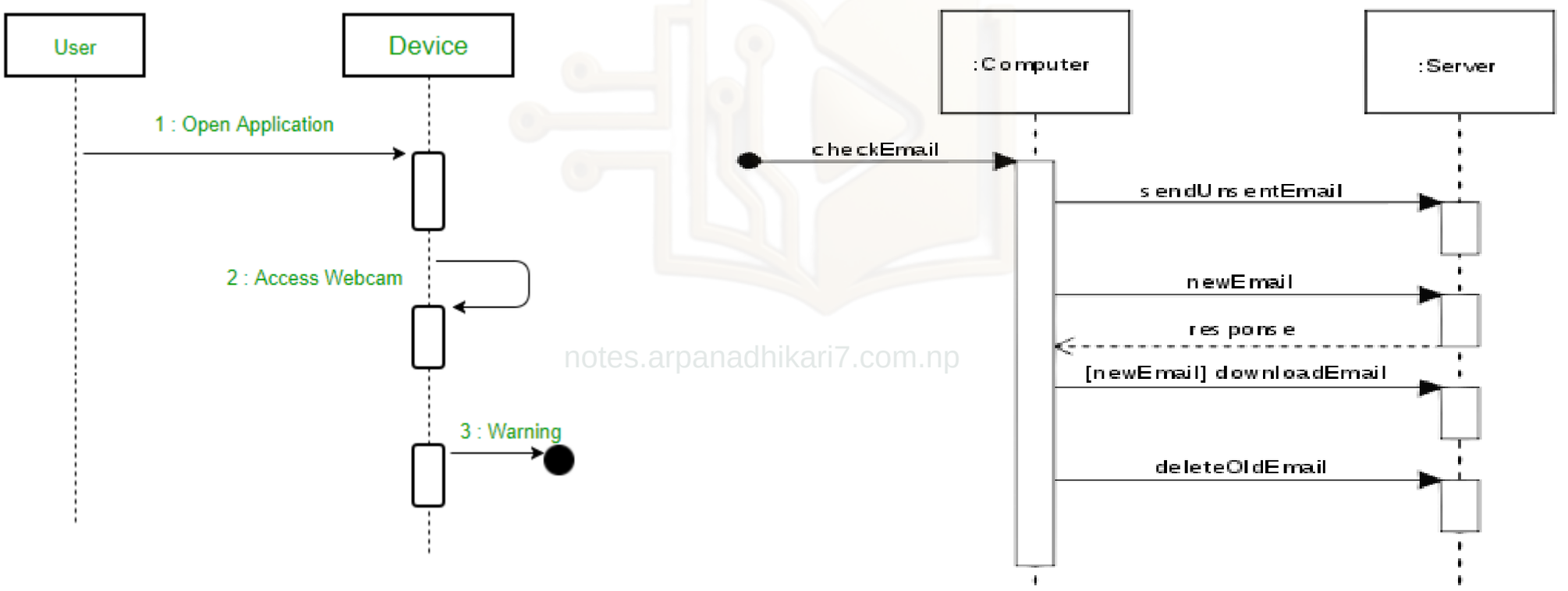
- Objects or participants when no longer needed can also be deleted from a sequence diagram.
- This is done by adding an ‘X’ at the end of the lifeline of the said participant.



Available on notes.arpanadhikari7.com.np

Lost and Found Message

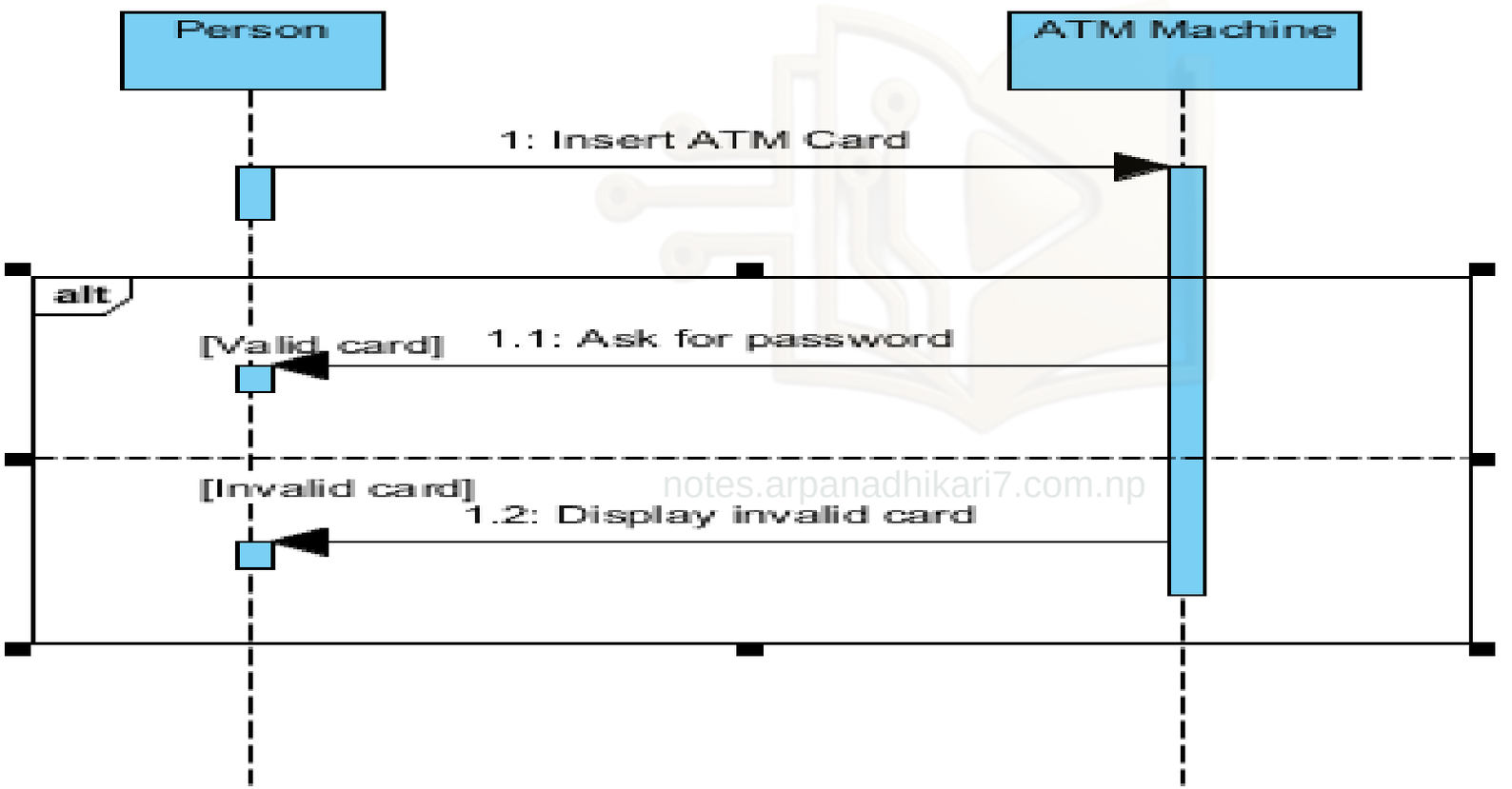
- A lost message is a message that has a known sender but the receiver is not known.
- A found message is a message that does not have a known sender but has a receiver.



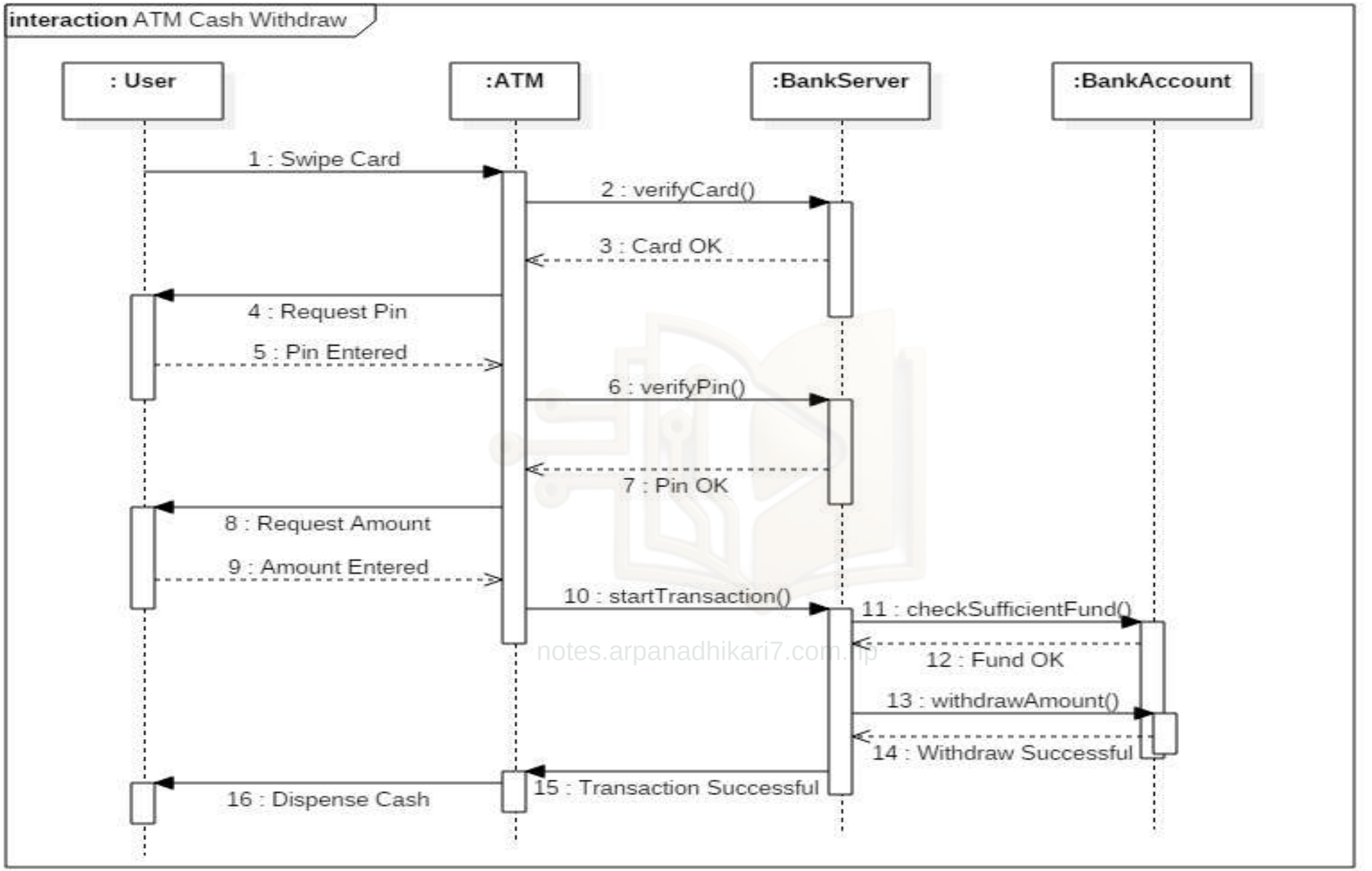
notes.arpanadhikari7.com.np

Alternate Combined fragment In SSD

- An alternative combined fragment is used to specify an area of a group of lifelines or actors to show **conditional flow** in a sequence diagram.
- It also models the if-then-else logic in the sequence diagram.



notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

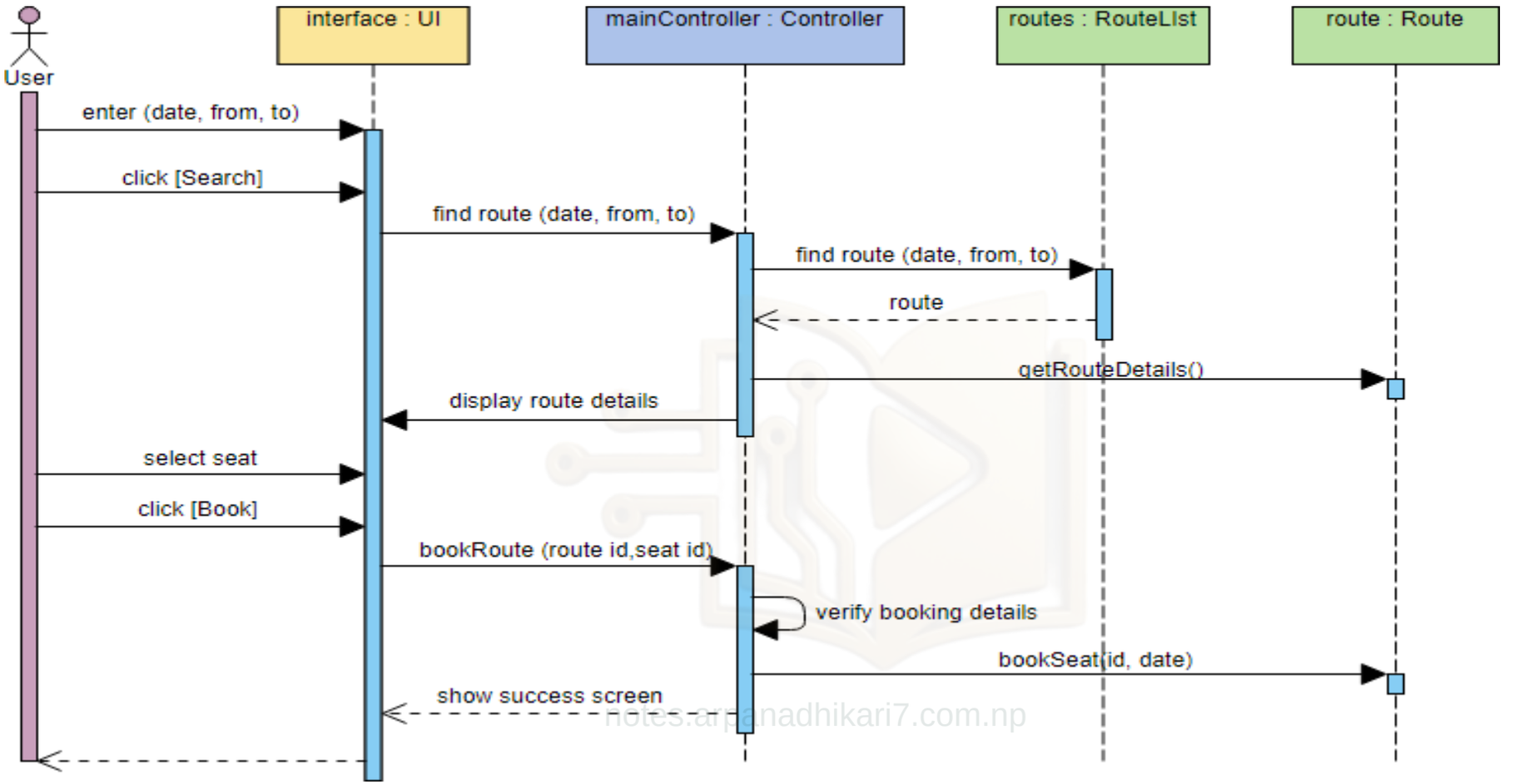


Figure: Online Ticket Booking System

Available on notes.arpanadhikari7.com.np

Communication/Collaboration Diagrams

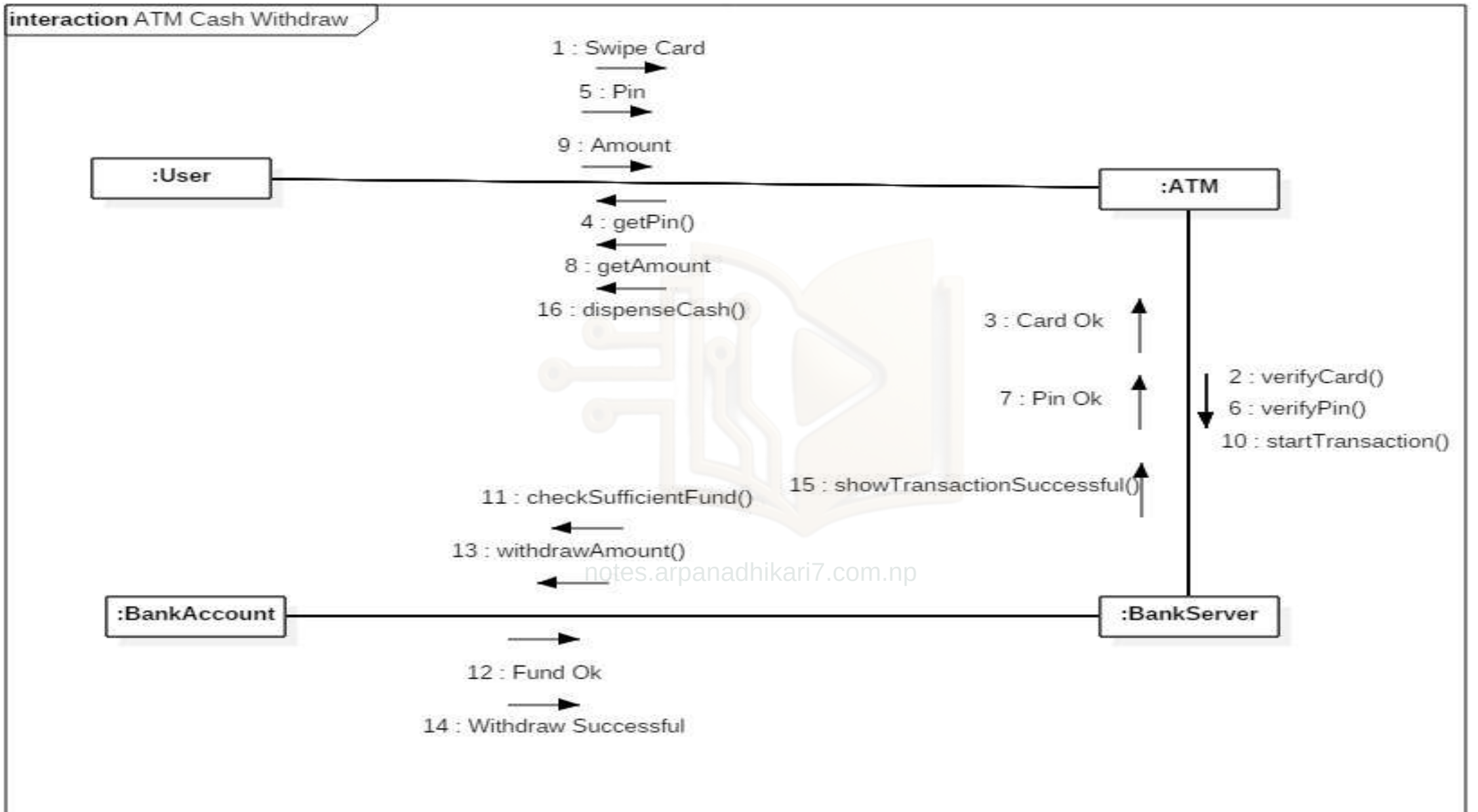
- A communication diagram is an illustration of the relationships and interactions among software objects in the Unified Modeling Language (UML).
- In this diagram, an object is also called a collaborator.
- The behavioral aspect is described by the set of messages exchanged among the different collaborators.
- The use of the collaboration diagrams in our development process would be to help us to determine which classes are associated with which other classes.
- Collaboration diagrams have two main features:

1. There is the path to indicate how one object is linked to another.
2. There is the sequence number to indicate the time order of a message.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Collaboration Diagram for ATM System



notes.arpanadhikari7.com.np

Activity Diagram

- It is a behavioral diagram that illustrates the flow of activities through a system.
- They can be used to examine business processes in order to identify its flow and requirements.
- An activity diagram shows the flow from activity to activity.
- The execution of an activity ultimately expands into the execution of individual actions, each of which may change the state of the system or communicate messages.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Notations Used in Activity Diagram

1. Start Node Notation



2. Activity Notation



3. Control Flow Notation



4. Final Node Notation



notes.arpanadhikari7.com.np

5. Decision Node Notation



6. Fork Notation

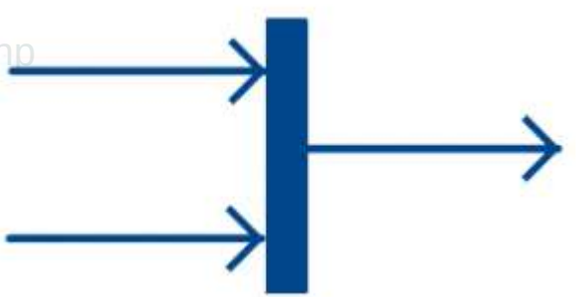
➢Used to represent a flow that may branch into two or more parallel flows.



notes.arpanadhikari7.com.np

7. Join Notation

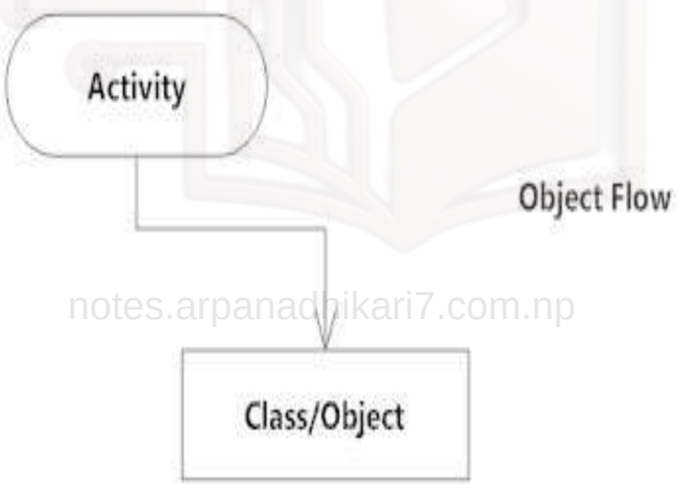
➢Used to combine two or more parallel flows.



Available on notes.arpanadhikari7.com.np

8. Object Flow

- Object flow refers to the creation and modification of objects by activities.
- An object flow arrow from an activity to an object means that the action creates or influences the object.
- An object flow arrow from an object to an activity indicates that the action state uses the object.

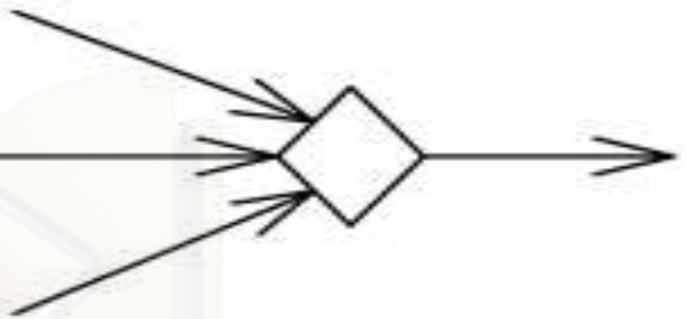


notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

9. Merge Node Notation

- Merge node is a control node that brings together multiple incoming alternate flows to accept single outgoing flow.



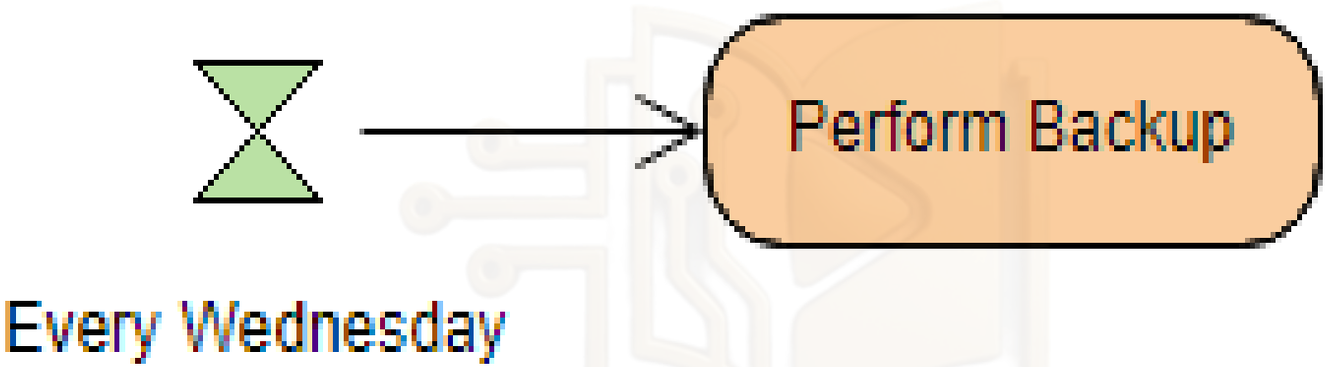
10. Exit point Notation



notes.arpanadhikari7.com.np

11. Time Event and Event Signal

➢Time Event refers to an event that stops the flow for a time; an hourglass depicts it.

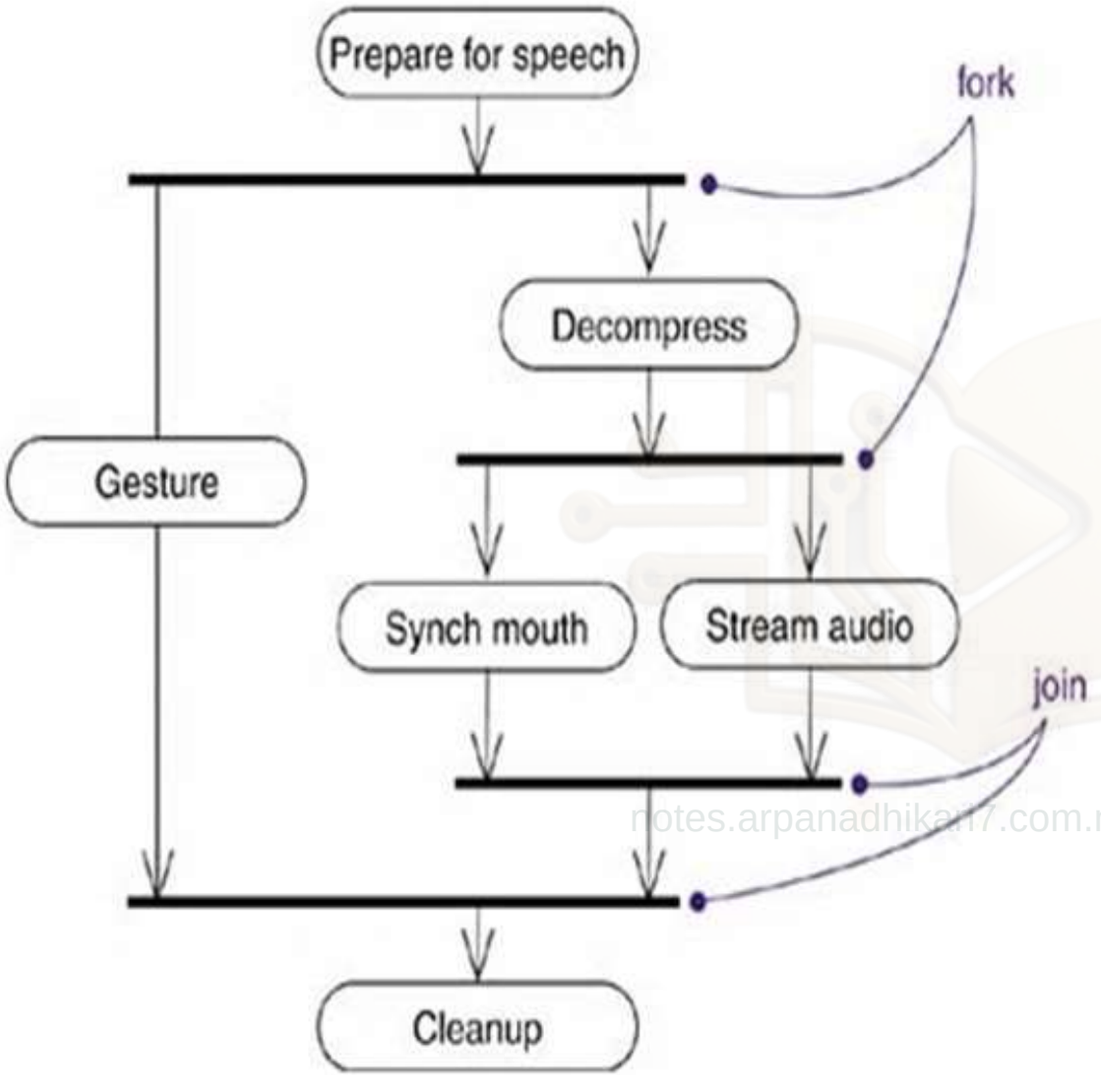


12. Guard

notes.arpanadhikari7.com.np

➢A guard is a condition that must be true in order to traverse a transition.

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Steps For Drawing Activity Diagram

1. **Figure out the action steps from the use case:** Here we need to identify the various activities and actions your business process or system is made up of.
2. **Identify the actors and objects involved:** If we already have figured out who the actors are, what the objects are then it's easier to identify and show each action they are responsible for.
3. **Find a flow among the activities:**

- Figure out in which order the actions are processed.
- Mark down the conditions that have to be met in order to carry out certain processes, which actions occur at the same time and whether you need to add any branches in the diagram.
- Also analyze do we have to complete some actions before we can proceed to others or not.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

4. Add swimlanes:

- Here, we have already figured out who and which object is responsible for each action.
- Now it’s time to assign them a swimlane and group each action they are responsible for under them.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

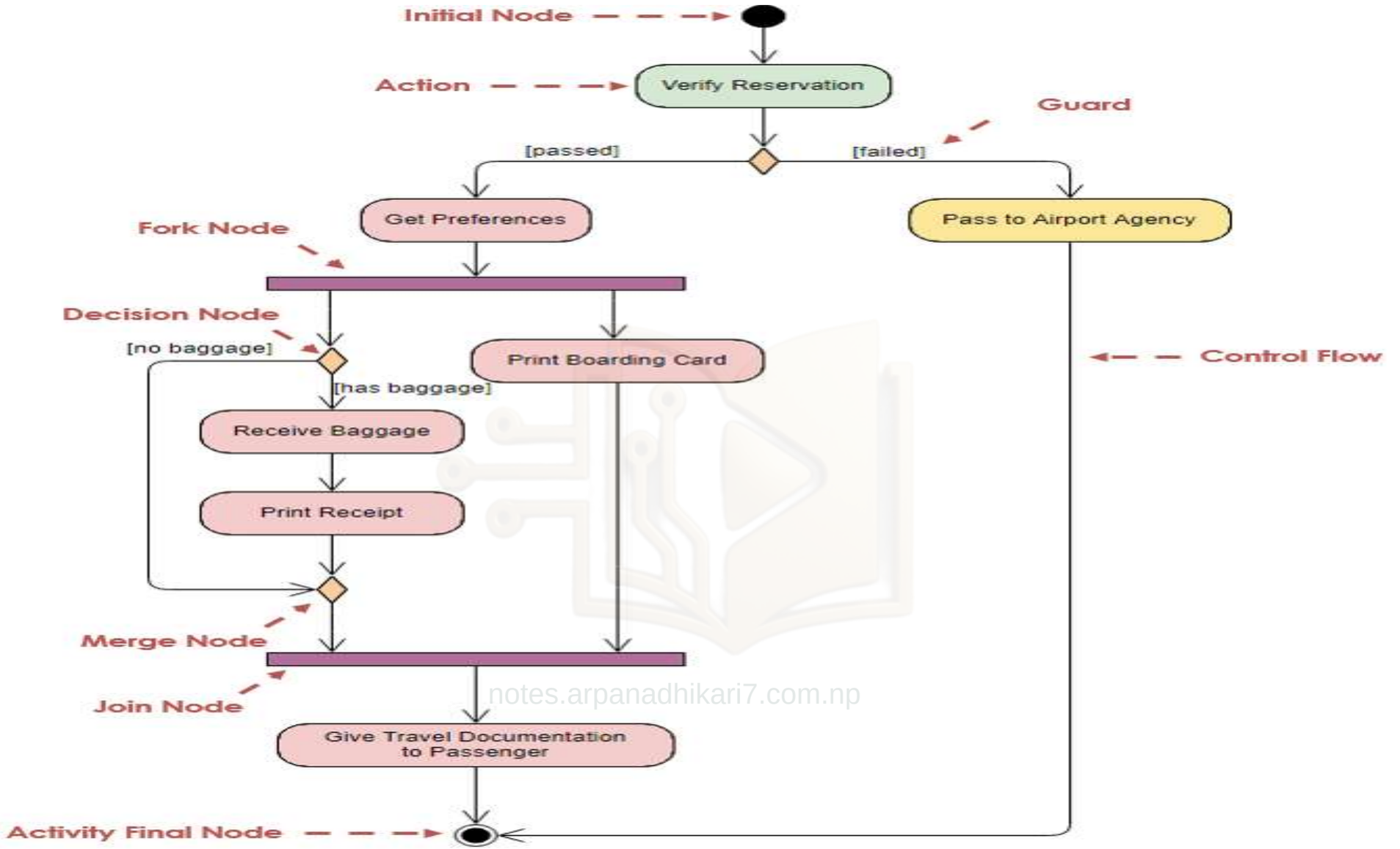


Figure: Activity Diagram Example (Without Swimlane)

Available on notes.arpanadhikari7.com.np

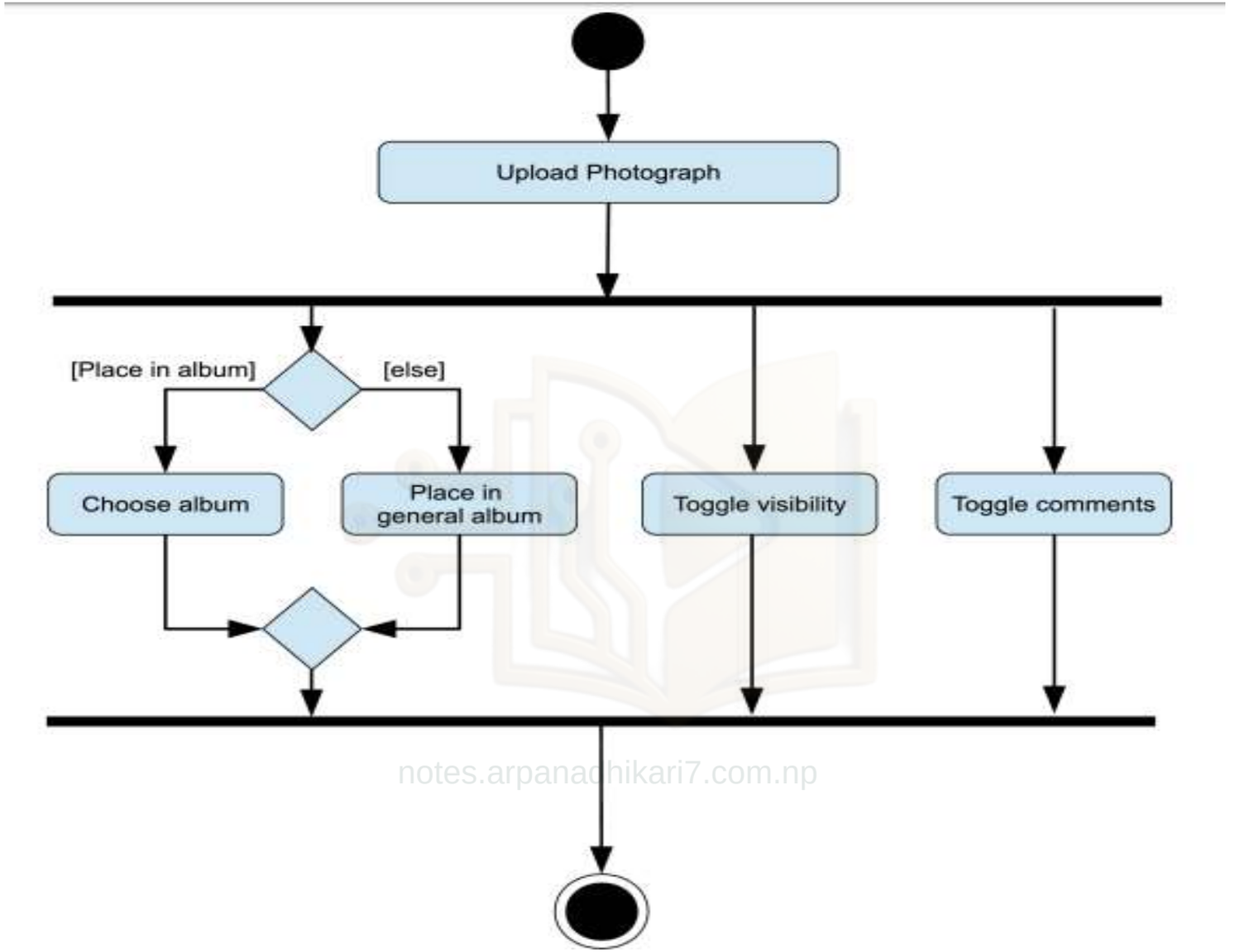


Figure: Activity Diagram Example (Without Swimlane)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Swimlanes In Activity Diagram

- In activity diagrams swimlanes – also known as partitions – are used to represent or group actions carried out by different objects in a single thread.
- Swimlanes add an extra level of clarity about who does what in Activity Diagrams.
- In an activity diagram partitioned into swimlanes, every activity belongs to exactly one swimlane, but transitions may cross lanes.
- Activity Diagram with swimlanes is also called cross-functional diagram (swimlanes are sometimes called functional bands).
- The swim lane flowchart differs from other flowcharts in that processes and decisions are grouped visually by placing them in lanes.

notes.arpanadhikari7.com.np

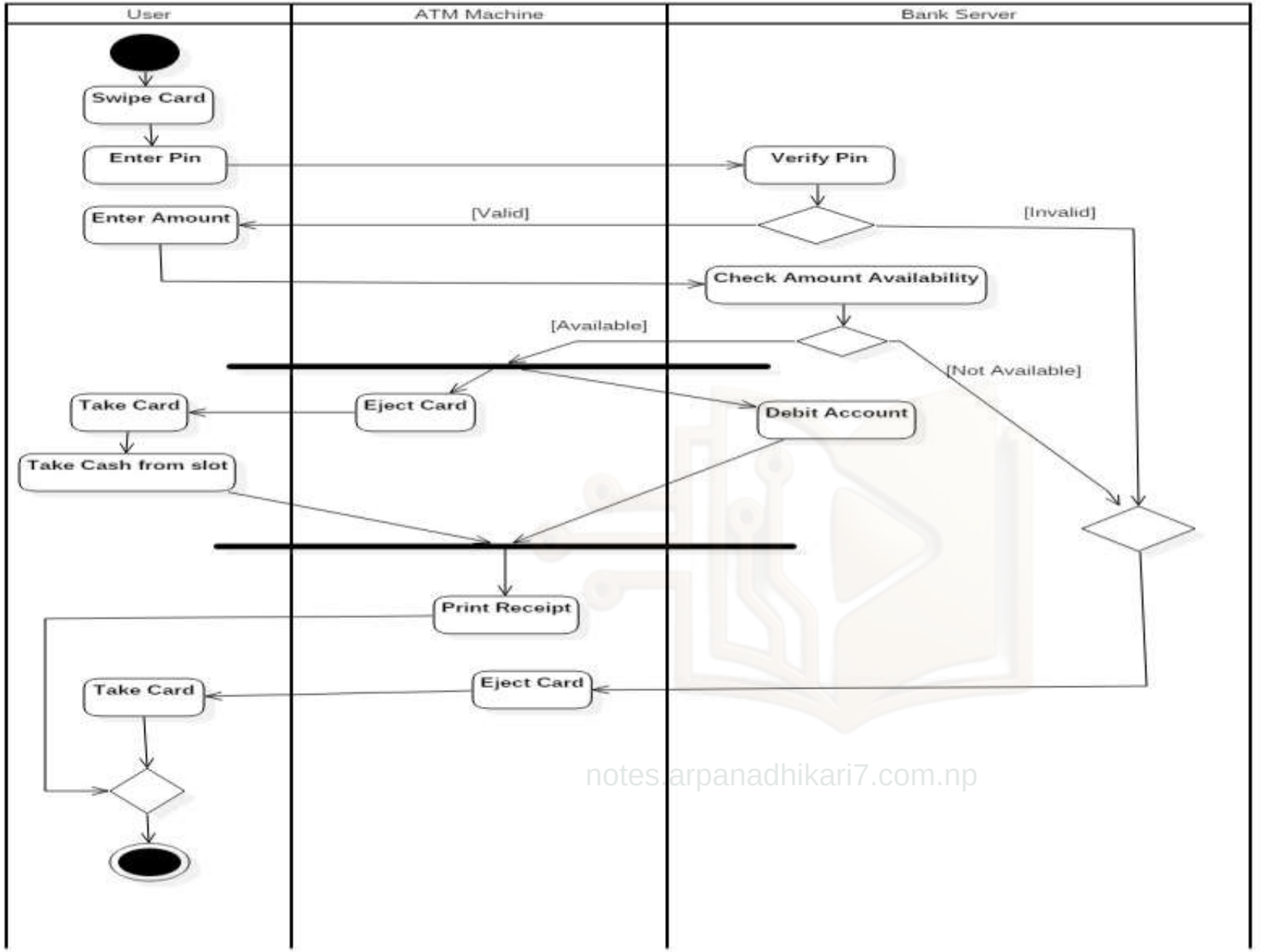
Available on notes.arpanadhikari7.com.np

Swimlane purposes and benefits

- Swimlane Diagrams, and swimlanes used in other diagram types, highlight which process steps or sub-processes are assigned to a particular actor in the organization.
- We can highlight redundancies between different lanes and identify bottlenecks, waste and other inefficiencies.
- This sometimes reveals duplicative or unnecessary steps in a process.
- It also can highlight process delays or capacity constraints within a particular swimlane so that they can be addressed and resolved.
- Swimlanes can increase performance and quality and reduce unnecessary work and costs.
- Swimlane Diagrams can communicate in clearer terms than a narrative description would.

notes.arpanadhikari7.com.np

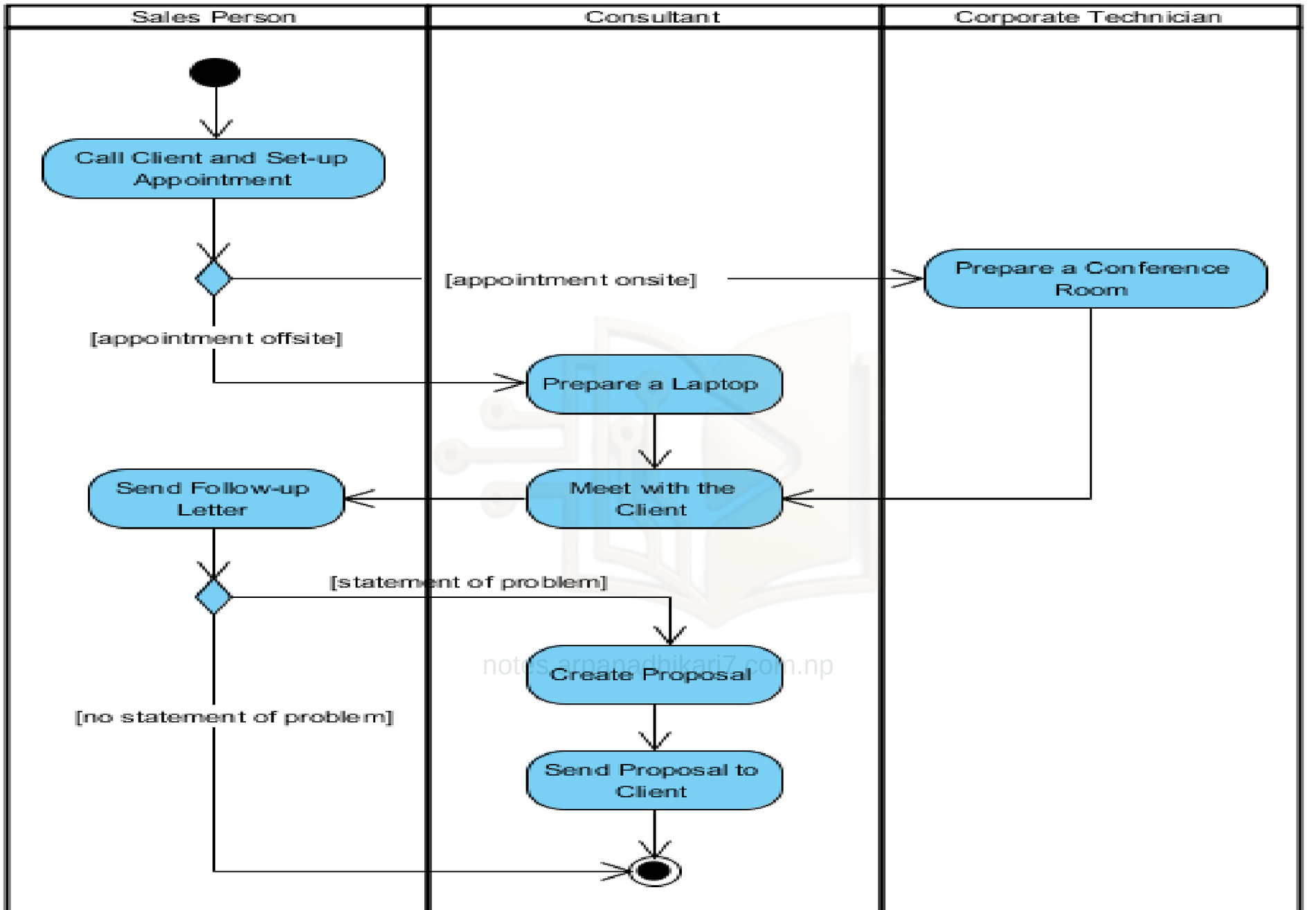
Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Figure: Activity Diagram of ATM

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Fork and Join

- A fork may have one incoming transition and two or more outgoing transitions, each of which represents an independent flow of control.
- In the join, the activities associated with each of the paths continues in parallel. At the join, the concurrent flows synchronize, meaning that each waits until all incoming flows have reached the join, at which point one flow of control continues on below the join.
- **NOTE: <u>Joins and forks should balance</u>**, meaning that the number of flows that leave a fork should match the number of flows that enter its corresponding join.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Structural Model

- Structural model is a model or view of a system that emphasizes the structure of the objects, including their relationships, attributes and operations.
- Structural model represents the framework for the system and this framework is the place where all other components exist.
- Static Model represents the static aspects of a software system.
- The structural model never describes the dynamic behavior of the system.
- The time and cost for developing a system using structural model is comparatively high as compared to behavioral model.
- Diagrams used for structural modeling are class diagram, object diagram, component diagram, deployment diagram, package diagram, profile diagram, etc.

Available on notes.arpanadhikari7.com.np

Class Diagram

- A class diagram is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects.
- Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.
- Class diagram describes the attributes and operations of a class and also the constraints imposed on the system.
- A UML class diagram is made up of a set of classes and a set of relationships between classes.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Purpose of Class Diagram

- To model the static view of an application.
- To describe responsibilities of a system.
- Business Analysts can use class diagrams to model systems from business perspective.
- It reduces thousands of words of explanation in a few graphical diagrams that may reduce time consumption to understand.
- It makes communication more clear and real.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

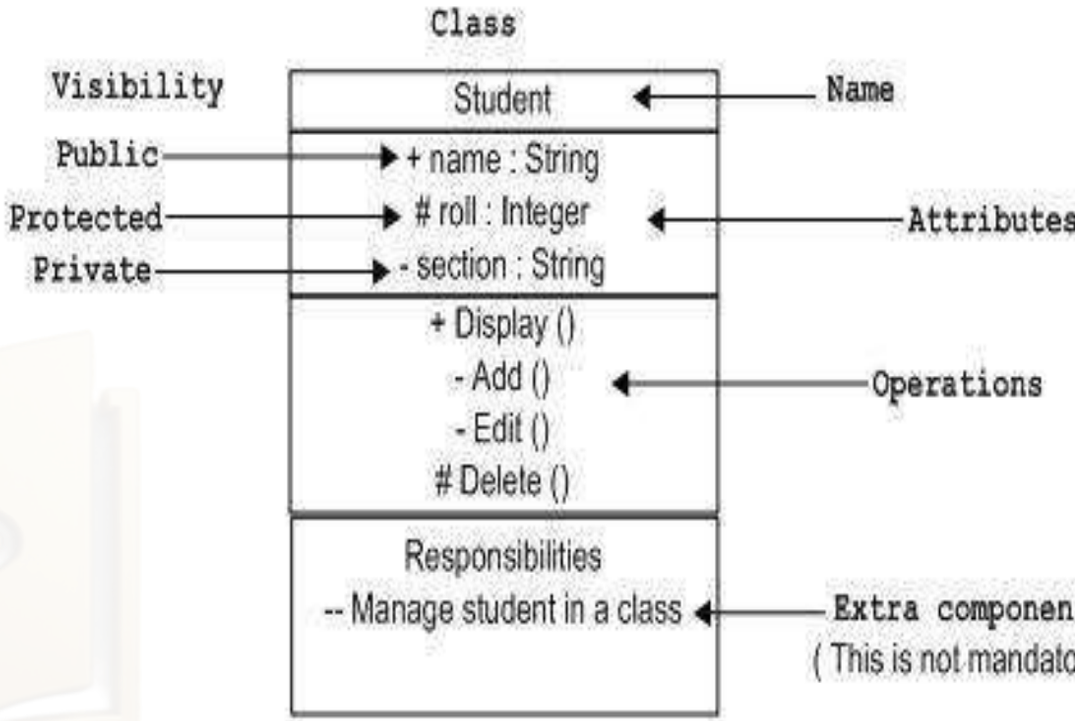
Points to remember while drawing a class diagram

1. The name of the class diagram should be meaningful to describe the aspect of the system and should start with capital letter.
2. Each element and their relationships should be identified in advance.
3. Responsibility (attributes and methods) of each class should be clearly identified.
4. For each class, minimum number of properties should be specified, as unnecessary properties will make the diagram complicated.
5. Use notes whenever required to describe some aspect of the diagram. At the end of the drawing it should be understandable to the developer/coder.
6. Finally, before making the final version, the diagram should be drawn on plain paper and reworked as many times as possible to make it correct.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Class Notation



- The diagram is divided into four parts.
- The top section is used to name the class. The first letter of class name is capital.
- The second one is used to show the attributes of the class.
- The third section is used to describe the operations performed by the class.
- The fourth section is optional to show any addi[illegible]
- - symbol represents that attributes or operations are private.
- + symbol represents that attributes or operations are public.
- # symbol represents that attributes or operations are protected.
- ~ symbol represents that attributes or operations are package.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Relationships in Class Diagram

1. Association(Directed/Undirected/Reflexive)
2. Multiplicity
3. Aggregation
4. Composition
5. Generalization/Inheritance
6. Dependency
7. Realization

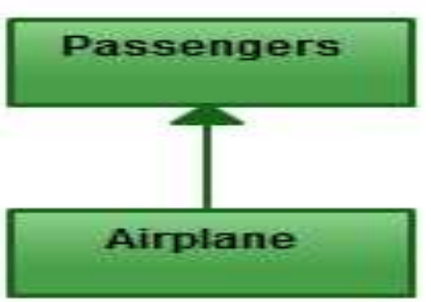
notes.arpanadhikari7.com.np

1. Association

- An association relationship is a structural relationship between two model elements that shows that objects of one classifier (actor, use case, class, interface, node, or component) connect and can navigate to objects of another classifier.
- In bidirectional relationships, an association connects two classifiers, the primary (supplier) and secondary (client) and is called binary associations. A binary association is represented by a line. An association that connects n number of classifiers is known as n-nary association.
- Association is represented by a solid line. Line can also feature multiplicity rules (eg. one-to-one, one-to-many, many-to-many) for the relationship.

notes.arpanadhikari7.com.np





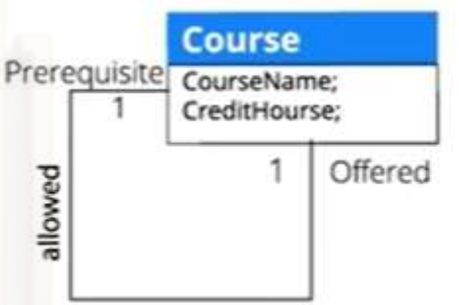
Directed Association

Available on notes.arpanadhikari7.com.np

2. Reflexive Association

- A reflexive association is an association from a class to itself.
- This relationship is shown when a class may have multiple functions or responsibilities.





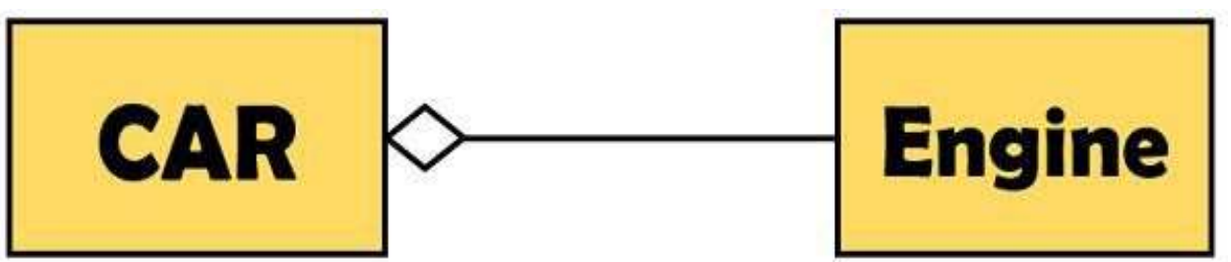


notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

3. Aggregation

- It is a type of association relationship where a child can exist independently of the parent.
- Example: Class (parent) and Student (child). Delete the Class and the Students still exist.
- The aggregation is a type of association which describes a **“has a”** type of relationship between the objects.
- Aggregation is represented as a solid line with a hollow diamond head.
- It is a weak type of association.
- The deletion of assembly doesn't affect its parts.

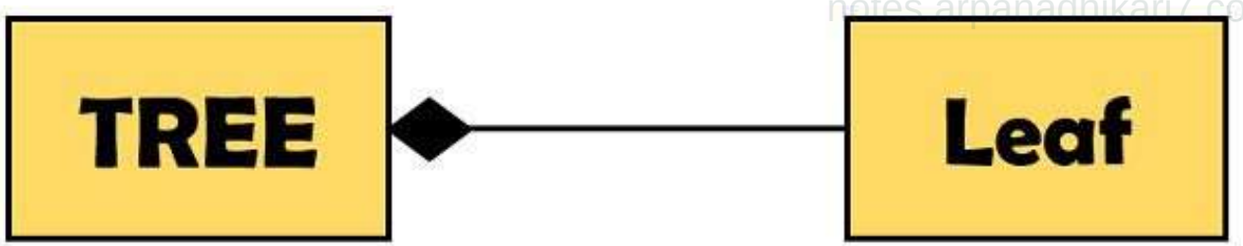


Available on notes.arpanadhikari7.com.np

4. Composition

- It is a type of association relationship where a child cannot exist independently of the parent.
- Example: House (parent) and Room (child). Rooms don't exist separate to a House.
- The composition is a type of association which describes a **"part of"** type of relationship between the objects.
- Composition is represented as a solid line with a solid diamond head.
- It is a strong type of association.
- The deletion of assembly does affect its parts. For example, a leaf is a part of a tree, if the tree is destroyed, then leaves are must be destroyed.

notes.arpanadhikari7.com.np





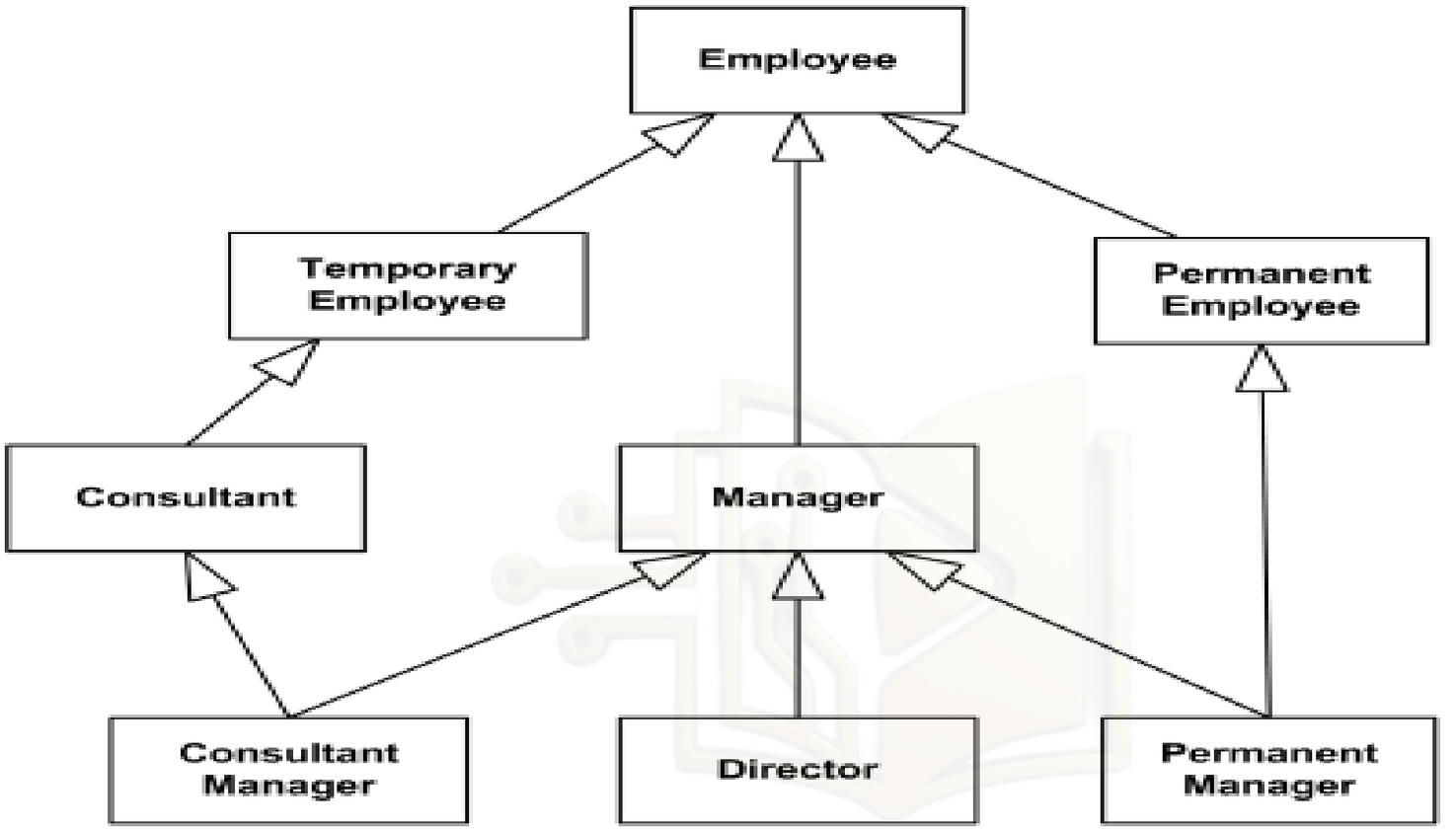
Available on notes.arpanadhikari7.com.np

5. Generalization or Specialization

- It is a relationship in which one model element (the child) is based on another model element (the parent).
- Generalization relationships are used in class, component, deployment, and use-case diagrams to indicate that the child receives properties, attributes, operations that are defined in the parent.
- Generalization relationship is also informally called "**Is A**" relationship.
- Generalization relationships do not have names.
- A generalization is shown as a line with a hollow triangle as an arrowhead between the symbols. The arrowhead points towards the parent entity or element.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



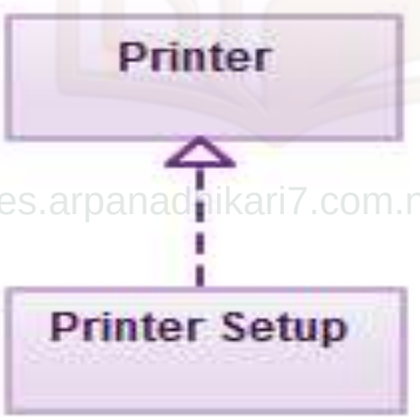
notes.arpanadhikari7.com.np

Figure: Generalization Relationship in UML

Available on notes.arpanadhikari7.com.np

6. Realization

- A realization relationship is a relationship between two model elements, in which one model element (the client) realizes the behavior that the other model element (the supplier) specifies.
- Several clients can realize the behavior of a single supplier.
- It denotes the implementation of the functionality defined in one class by another class. In other words, you can understand this as the relationship between the interface and the implementing class.



notes.arpanadhikari7.com.np

- To show the relationship in UML, a broken line with an unfilled solid arrowhead is drawn from the class that defines the functionality to the class that implements the function.
- In above example the printer setup interface or class is being implemented by the printer class.

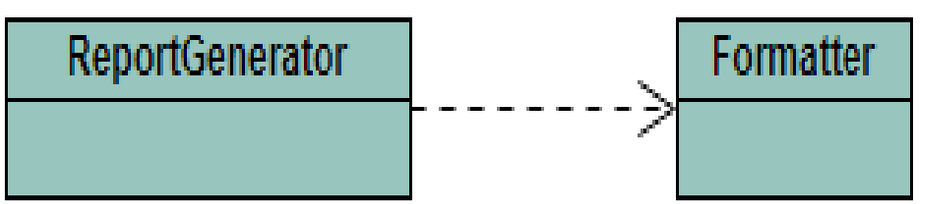
notes.arpanadhikari7.com.np

7. Dependency

- A dependency relationship is a relationship in which one element, the client, uses or depends on another element, the supplier.
- So changes made in one element(on which the other is depending) may affect the element which is dependent on the other.
- If a class A "uses" class B, then one or more of the following statements generally hold true:

- Class B is used as the type of a local variable in one or more methods of class A.
- Class B is used as the type of parameter for one or more methods of class A.
- Class B is used as the return type for one or more methods of class A.
- One or more methods of class A invoke one or more methods of class B.

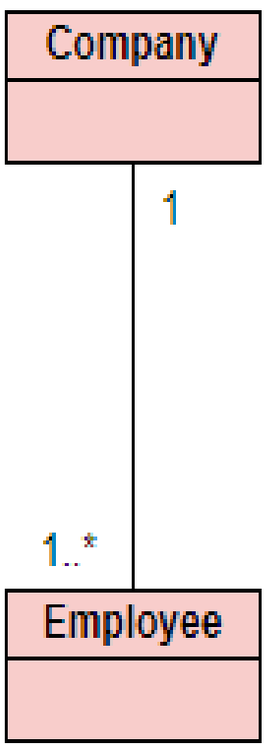
notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

8. Multiplicity

- Multiplicity in UML allows to specify cardinality - i.e. number of elements associated.



Multiplicities examples:

1	Exactly one, no more and no less
0..1	Zero or one
*	Many
0..*	Zero or many
1..*	One or many

notes.arpanadhikari7.com.np

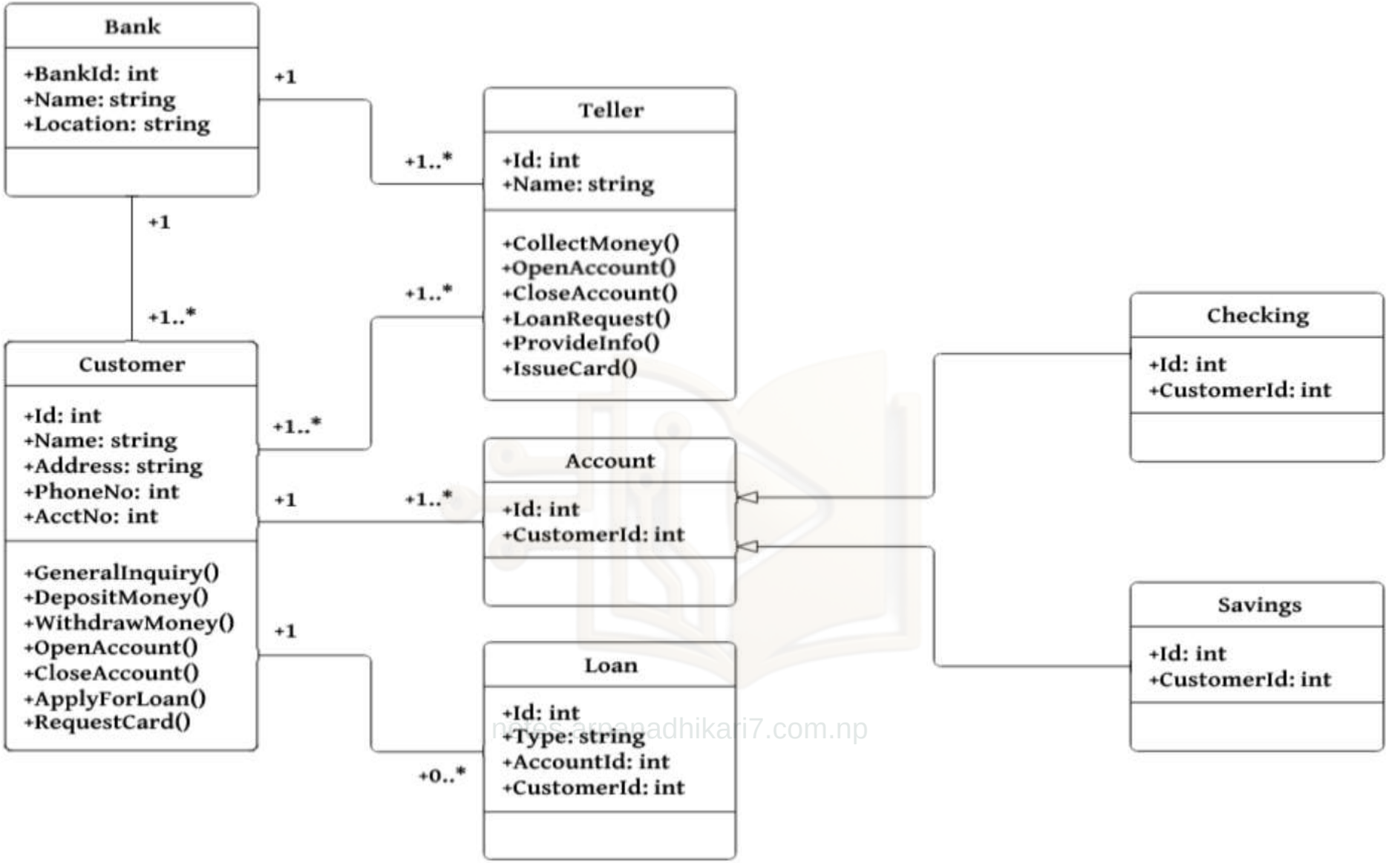


Figure: Class Diagram for Banking System

Available on notes.arpanadhikari7.com.np

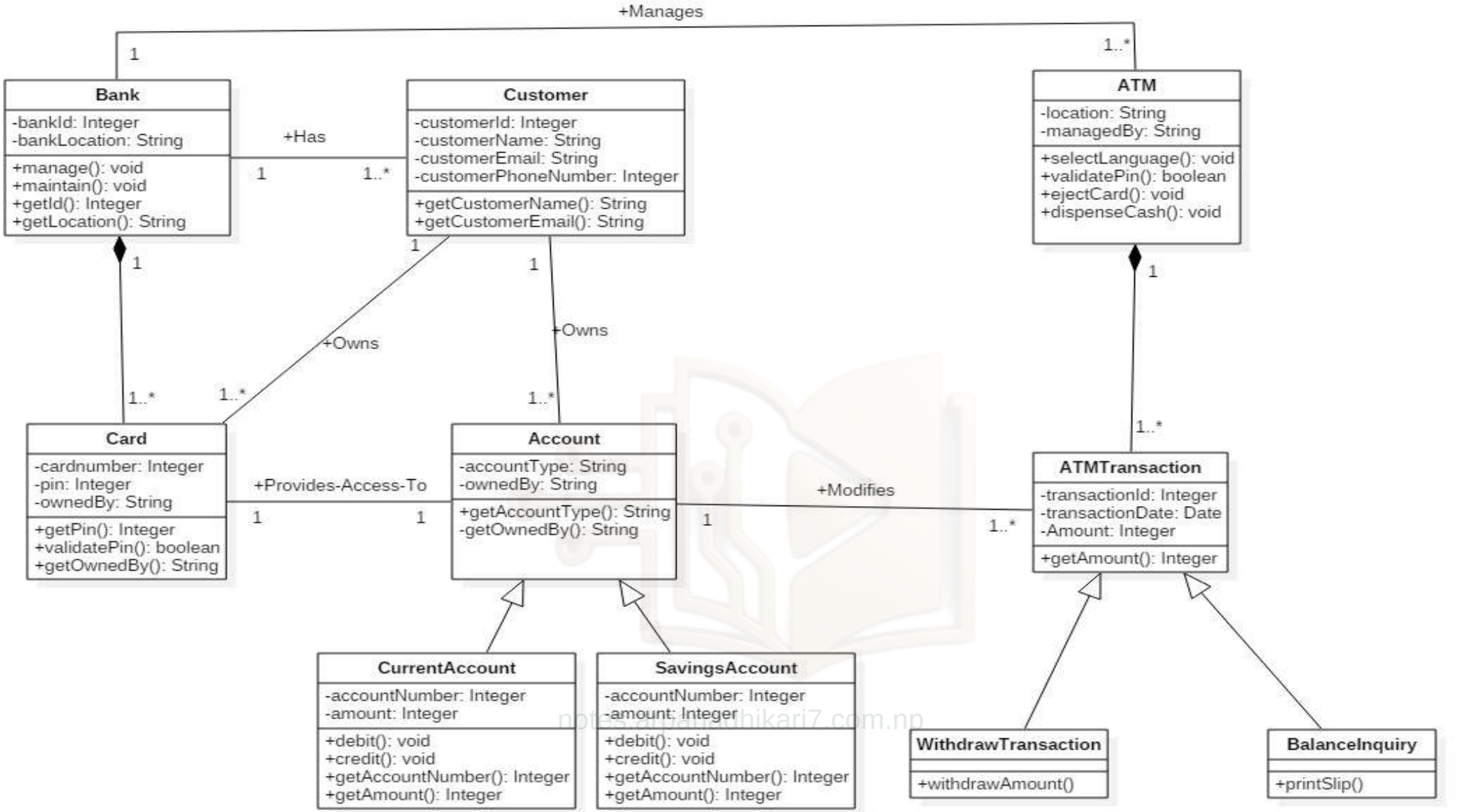
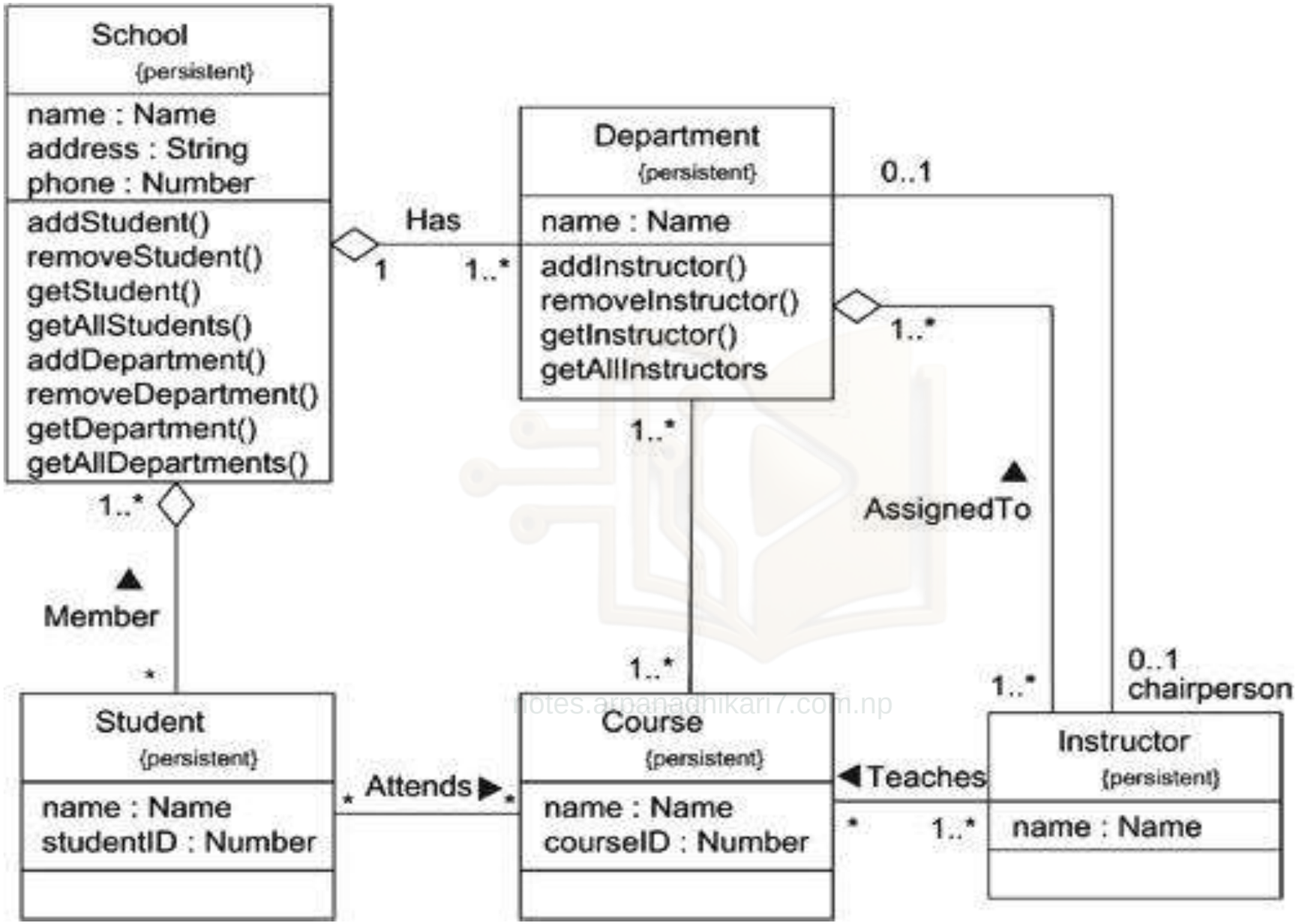


Figure: Class Diagram For ATM

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Object Diagram

- They show how a system will look like and relation of one object with the other at a given time.
- Object diagrams and class diagrams are closely related and use almost identical notation.
- Object diagrams are a set of objects and their relationship is just like class diagrams. They also represent the static view of the system.
- Object diagrams are derived from class diagrams so object diagrams are dependent upon class diagrams.
- An object diagram focuses on some particular set of objects and attributes, and the links between these instances.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Figure: Object Diagram Example

Available on notes.arpanadhikari7.com.np

Component Diagram

- A component diagram breaks down the actual system under development into various high levels of functionality.
- Component diagrams represent a set of components and their relationships. These components consist of classes, interfaces, or collaborations.
- During the design phase, software artifacts (classes, interfaces, etc.) of a system are arranged in different groups depending upon their relationship known as components.
- So it can be said component diagrams are used to visualize the implementation.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

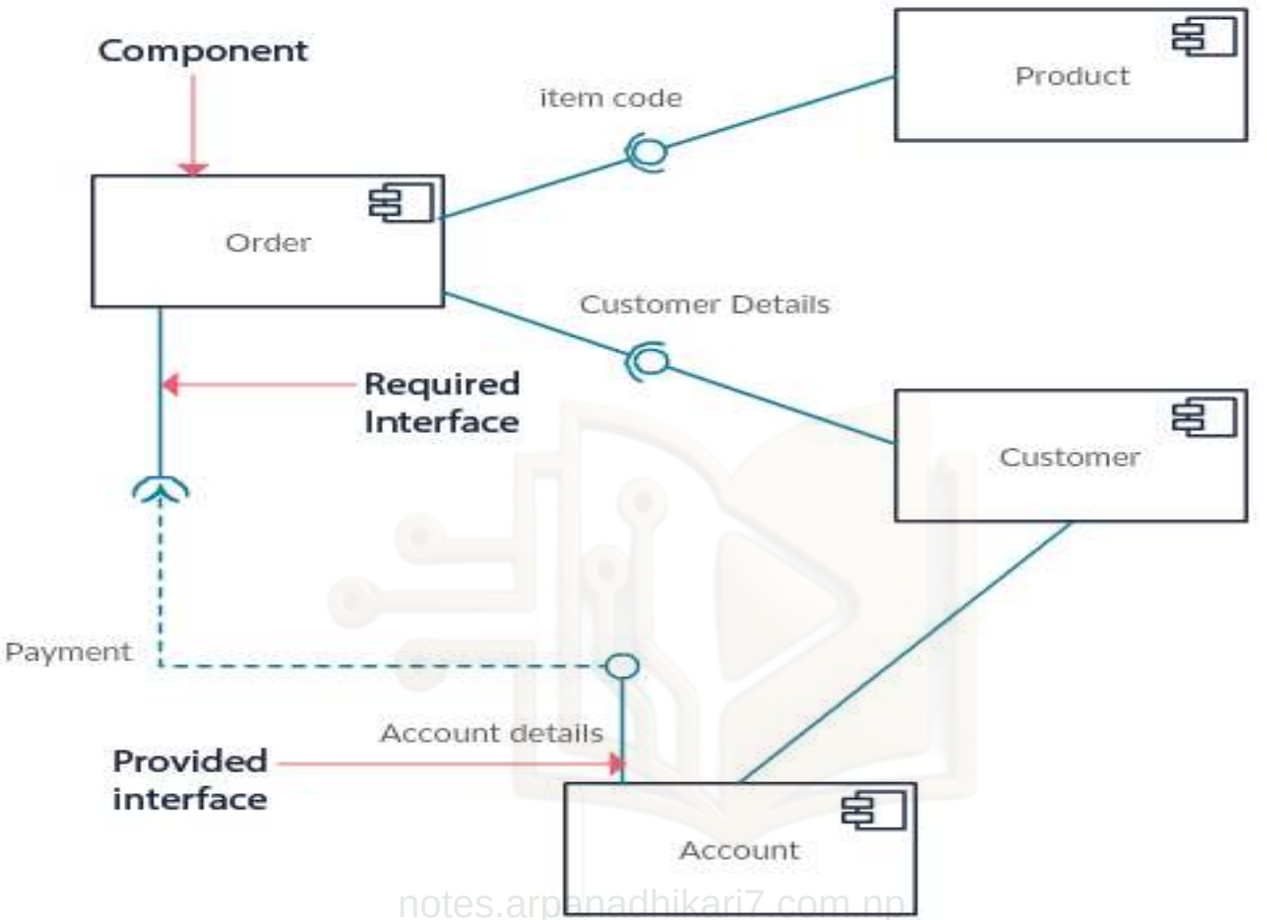


Figure: Component Diagram Example

Available on notes.arpanadhikari7.com.np

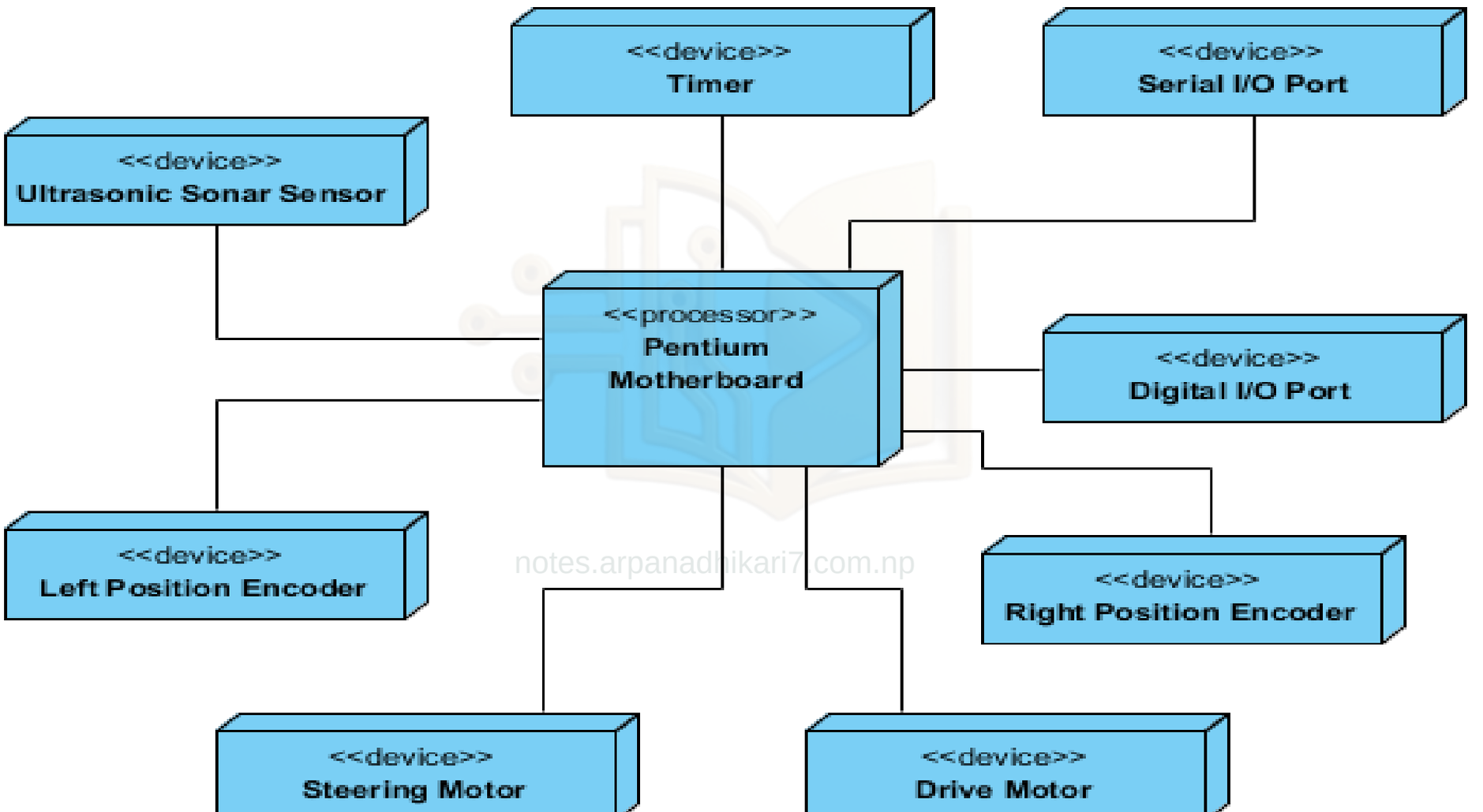
Deployment Diagram

- A deployment diagram is just a kind of class diagram, which focuses on a system's nodes.
- Deployment diagrams are used for describing the hardware components where software components are deployed.
- Graphically, a deployment diagram is a collection of vertices and arcs.
- Deployment diagrams are useful when your software solution is deployed across multiple machines with each having a unique configuration.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Deployment Diagram



notes.arpanadhikari7.com.np

State Machine Diagram

- A state machine diagram is a behavioral diagram that specifies the sequences of states an object goes through during its lifetime in response to events, together with its responses to those events.
- A state is a condition or situation during the life of an object during which it satisfies some condition, performs some activity, or waits for events.
- In the context of state machines, an event is an occurrence of a stimulus or action that can trigger a **state transition**.
- A transition is a relationship between two states indicating that an object in the first state will perform certain actions and enter the second state when a specified event occurs and specified conditions are satisfied.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

State

- A state is a constraint or a situation in the life cycle of an object, in which a constraint holds, the object executes an activity or waits for an event.
- Characteristics of State:

- State represent the conditions of objects at certain points in time.
- Objects (or Systems) can be viewed as moving from state to state
- A point in the lifecycle of a model element that satisfies some condition, where some particular action is being performed or where some event is waited.

- Guidelines:
  - State Names Should be Simple but Descriptive
  - Question "Black Hole" States. A black hole state is one that has transitions into it but none out.
  - Question "Miracle" States. A miracle state is one that has transitions out of it but none into it.

Available on notes.arpanadhikari7.com.np

Events

- Events appear in the internal transition compartment of a state or on a transition between states.
- An event may be one of four types:
  - Signal event - corresponding to the arrival of an asynchronous message or signal
  - Call event - corresponding to the arrival of a procedural call to an operation
  - Time event - a time event occurs after a specified time has elapsed
  - Change event - a change event occurs whenever a specified condition is met
- Characteristics of Events

- Represents incidents that cause objects to transition from one state to another.
- Internal or External Events trigger some activity that changes the state of the system and of some of its parts
- Events pass information, which is elaborated by Objects operations. Objects realize Events
- Design involves examining events in a state machine diagram and considering how those events will be supported by system objects

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Transitions and Actions

- A transition is a progression from one state to another and will be triggered by an event that is either internal or external to the entity being modeled.
- A transition is a progression from one state to another and will be triggered by an event that is either internal or external to the entity being modeled.



notes.arpanadhikari7.com.np

State Machine Diagram Notations

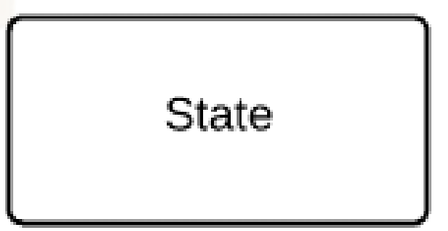
1. Initial/First State



2. Final State/ Terminator



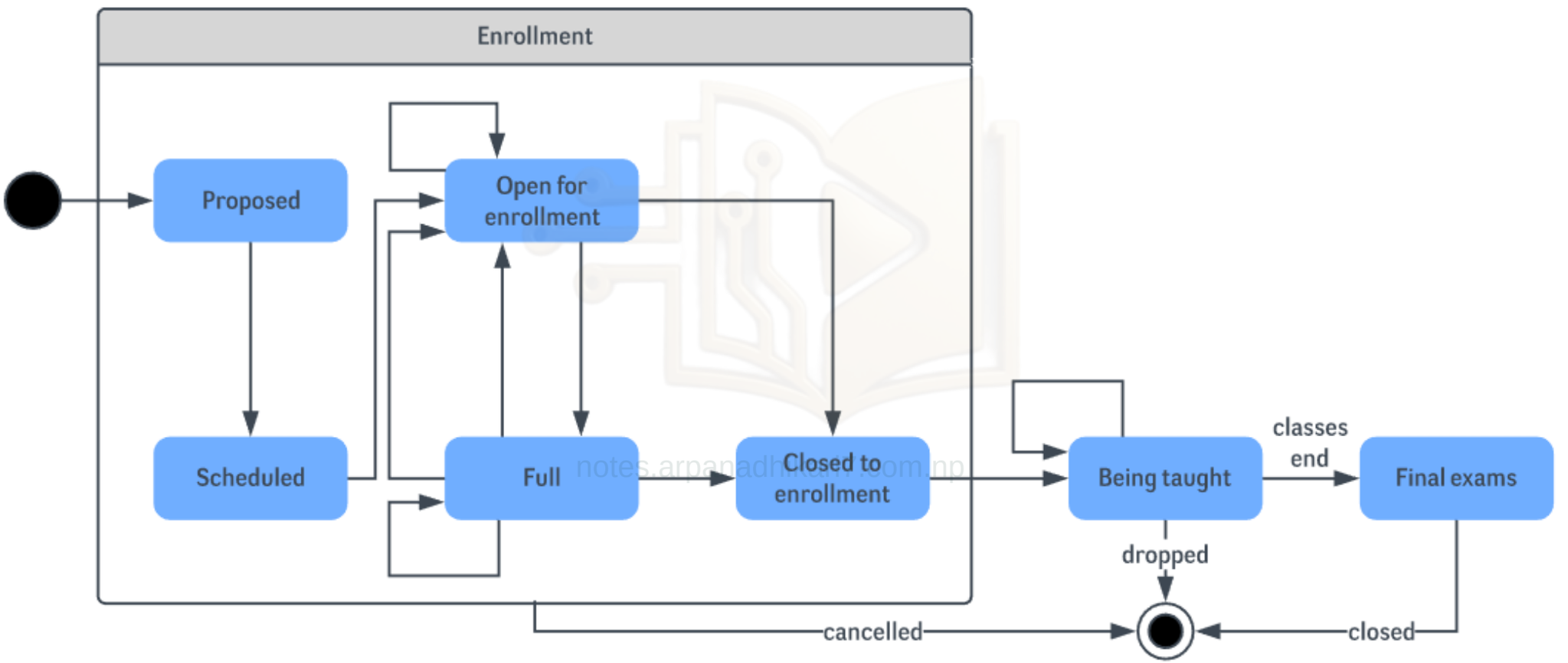
3. State



notes.arpanadhikari7.com.np

4. Sub-states

➢A state contained within a composite state's region.



Available on notes.arpanadhikari7.com.np

5. Guard

➢A guard is a condition that must be true in order to traverse a transition.

6. Decision Node Notation



7. Exit Point Notation

➢The exit point is typically used if the process is not completed but has to be escaped for some error or other issue.

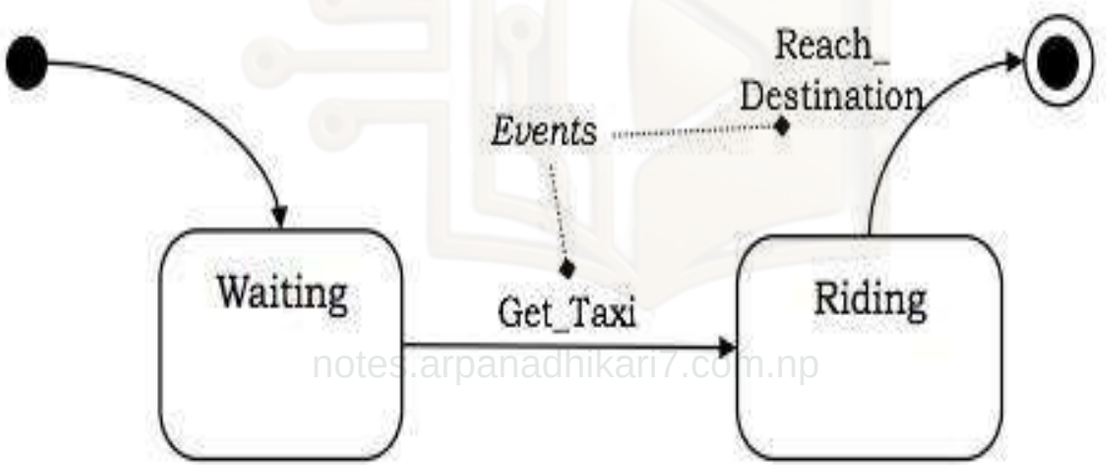
notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

Example

Suppose a person is taking a taxi from place X to place Y. The states of the person may be: Waiting (waiting for taxi), Riding (he has got a taxi and is travelling in it), and Reached (he has reached the destination). The following figure depicts the state transition.



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

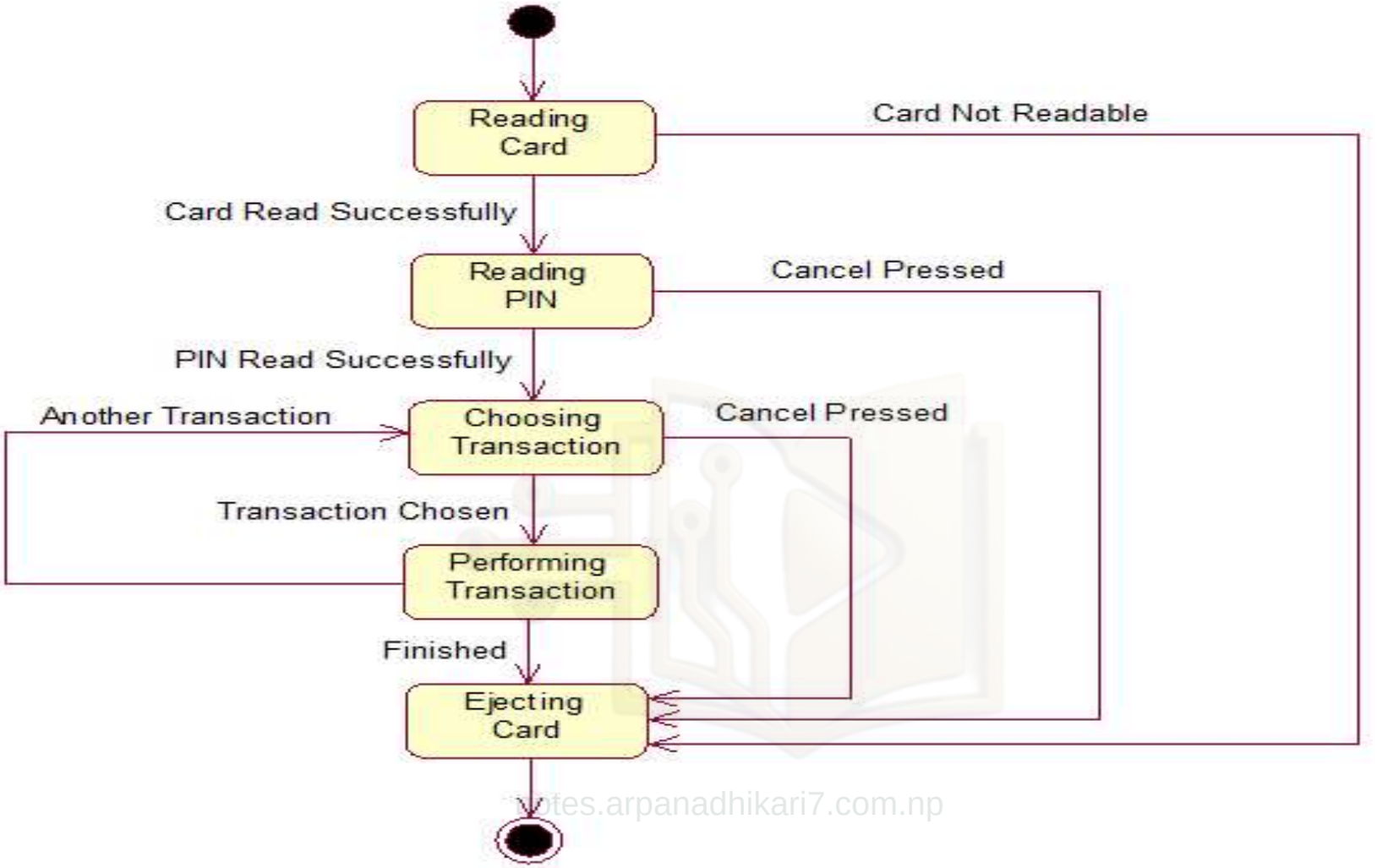


Figure: State Machine Diagram for ATM

Available on notes.arpanadhikari7.com.np

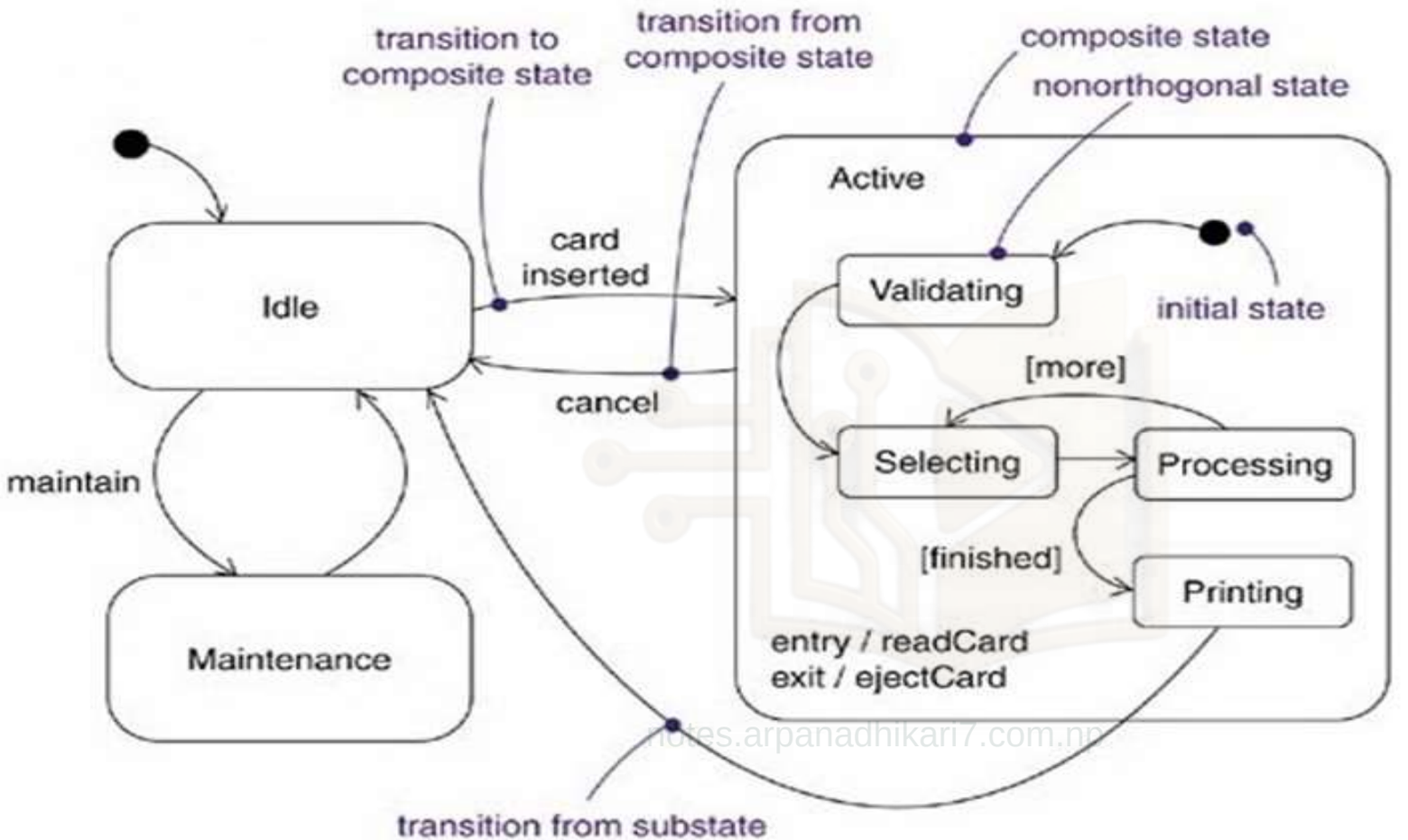


Figure: State Machine Diagram For ATM

Available on notes.arpanadhikari7.com.np